RAJALAKSHMI ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

DEPARTMENT OF INFORMATION TECHNOLOGY
LAB MANUAL

CS23432 – Software Construction

(REGULATION 2023)

RAJALAKSHMI ENGINEERING COLLEGE
Thandalam, Chennai-602015

Name: H.S.NIRTHYA THARA

Register No: 241001153

Year / Branch / Section: II - IT

Semester: IV

Academic Year: 2025-2026

INDEX

GITHUB QR-Bloggonut

EXP NO: 1	AZURE DEVOPS ENVIRONMENT SETUP

Aim:

To set up and access the Azure DevOps environment by creating an organization through the Azure portal.

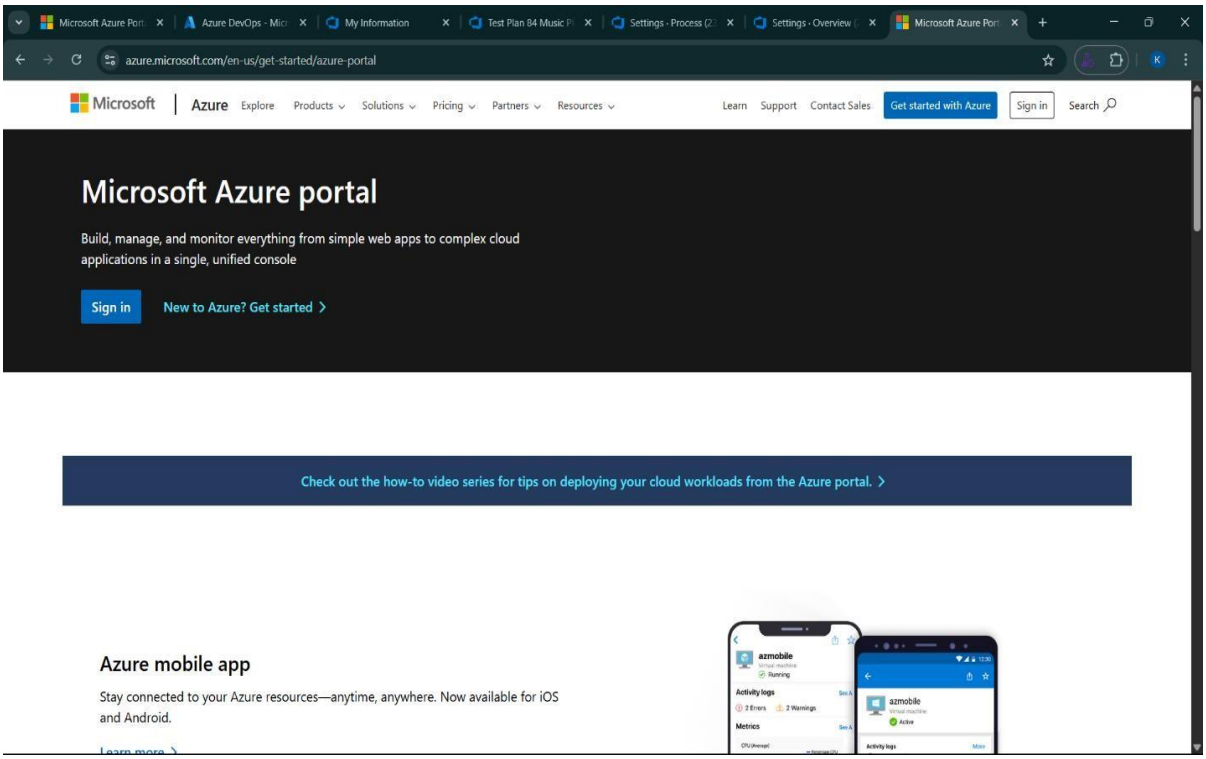
INSTALLATION:

1.Open your web browser and go to the Azure website:

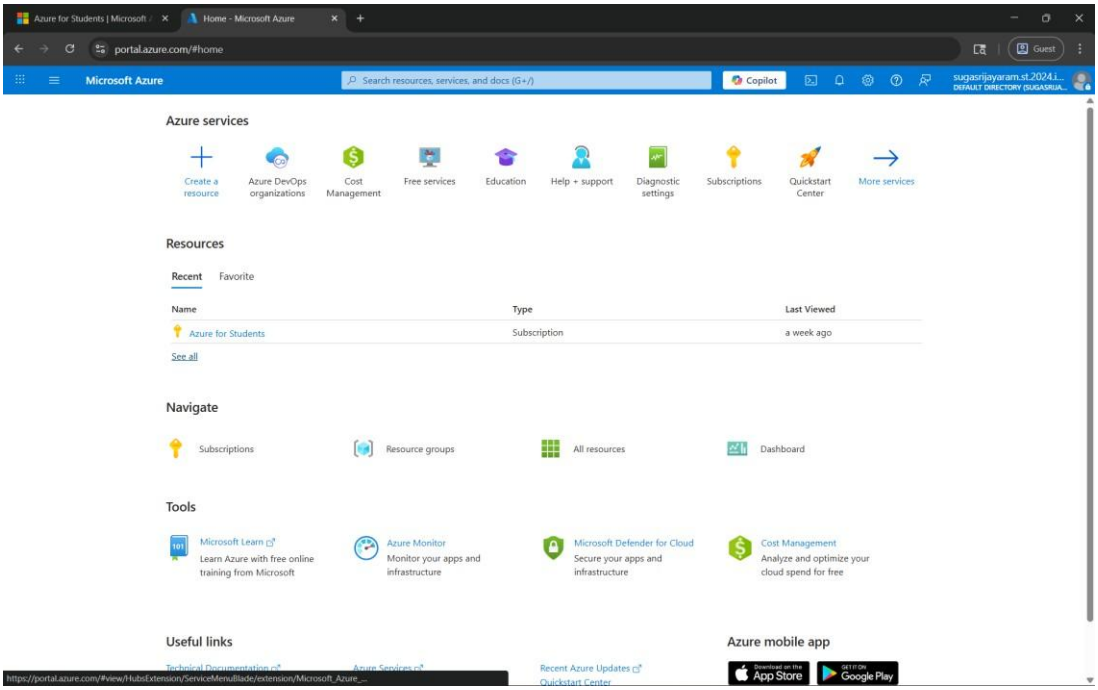
https://azure.microsoft.com/en-us/get-started/azure-portal.

Sign in using your Microsoft account credentials.

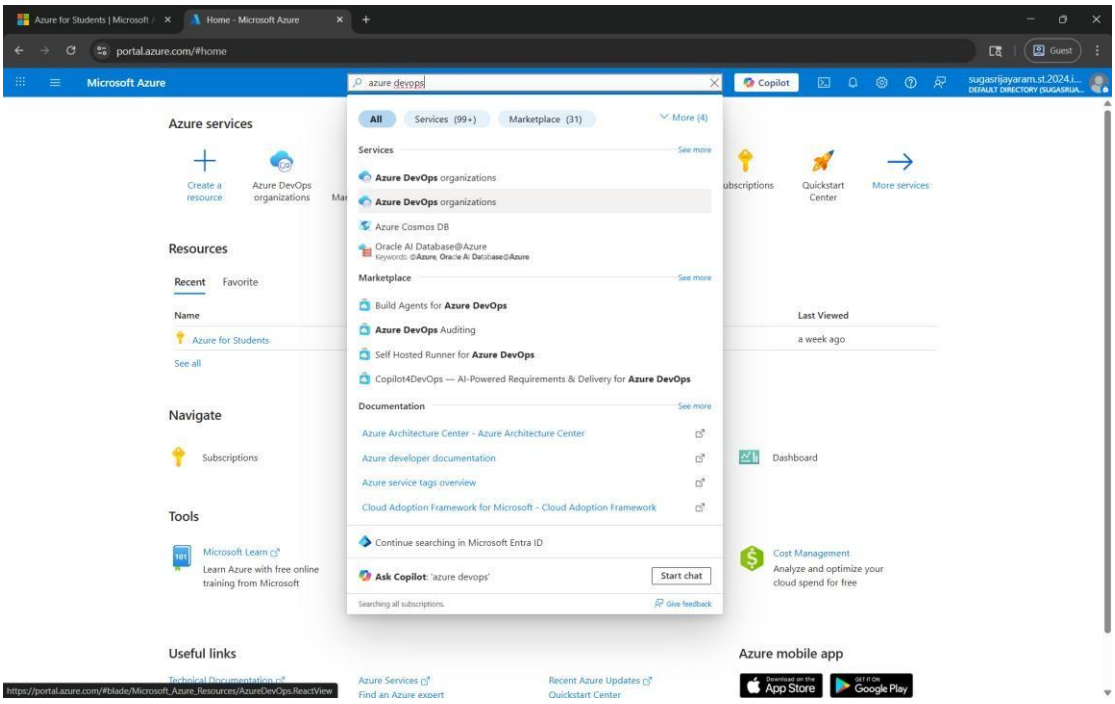
If you don't have a Microsoft account, you can create one here: https://signup.live.com/?lic=1



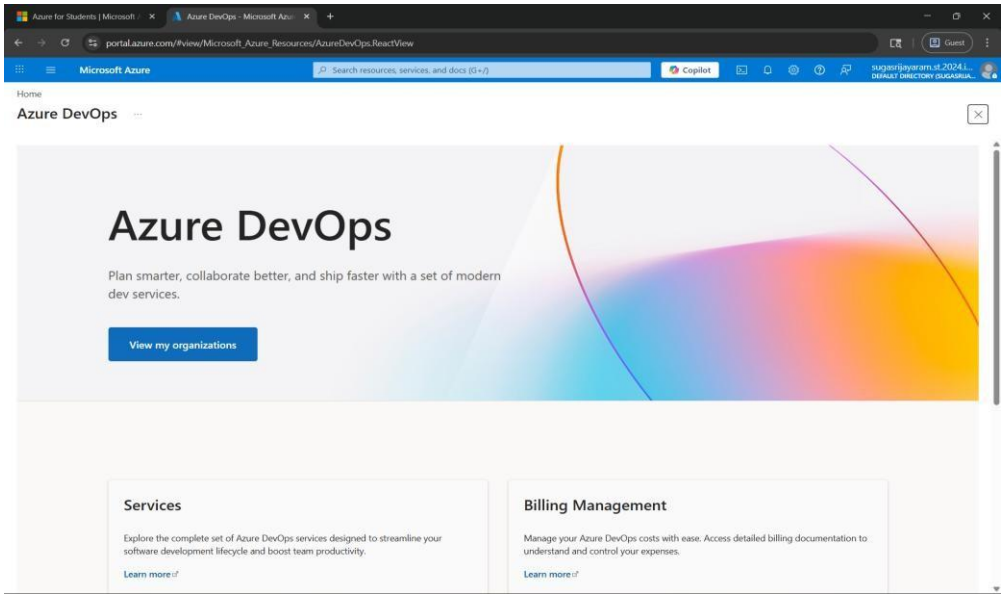
2.Azure home page

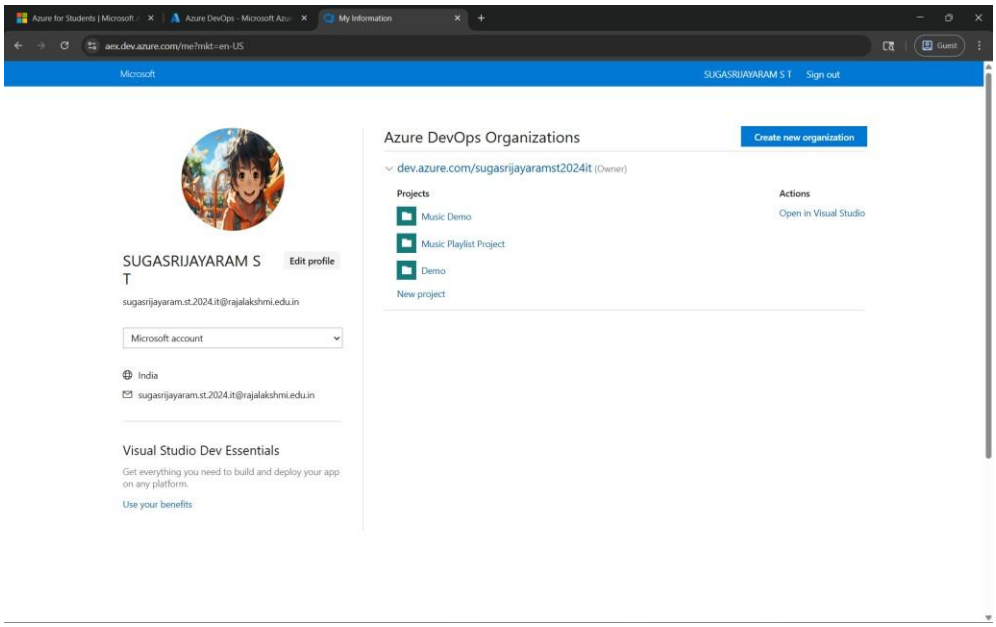


3.Open DevOps environment in the Azure platform by typing ***Azure DevOps Organizations*** in the search bar.



4.Click on the ***View My Organization*** link and create an organization and you should be taken to the Azure DevOps Organization Home page.





Result:

Successfully accessed the Azure DevOps environment and created a new organization through the Azure portal.

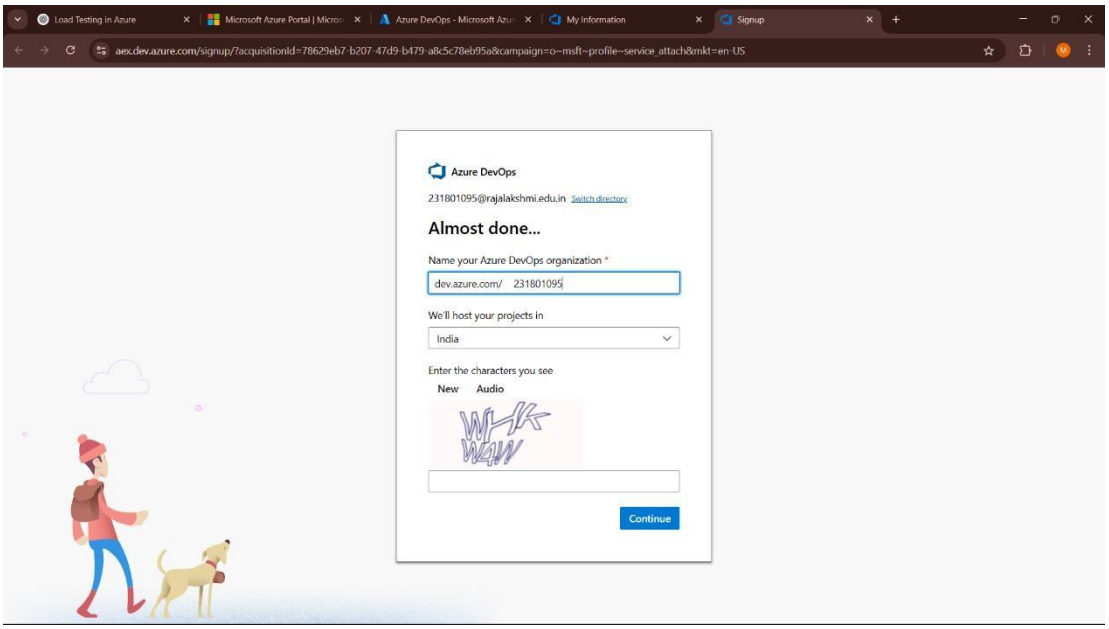
EXP NO: 2	AZURE DEVOPS PROJECT SETUP AND USER STORY MANAGEMENT

Aim:

To set up an Azure DevOps project for efficient collaboration and agile work management.

Procedure:

1.Create An Azure Account



2.Create the First Project in Your Organization

a. After the organization is set up, you'll need to create your first **project**. This is where you'll begin to manage code, pipelines, work items, and more.

b. On the organization's **Home page**, click on the **New Project** button.

c. Enter the project name, description, and visibility options:

Name: Blog Management Project.

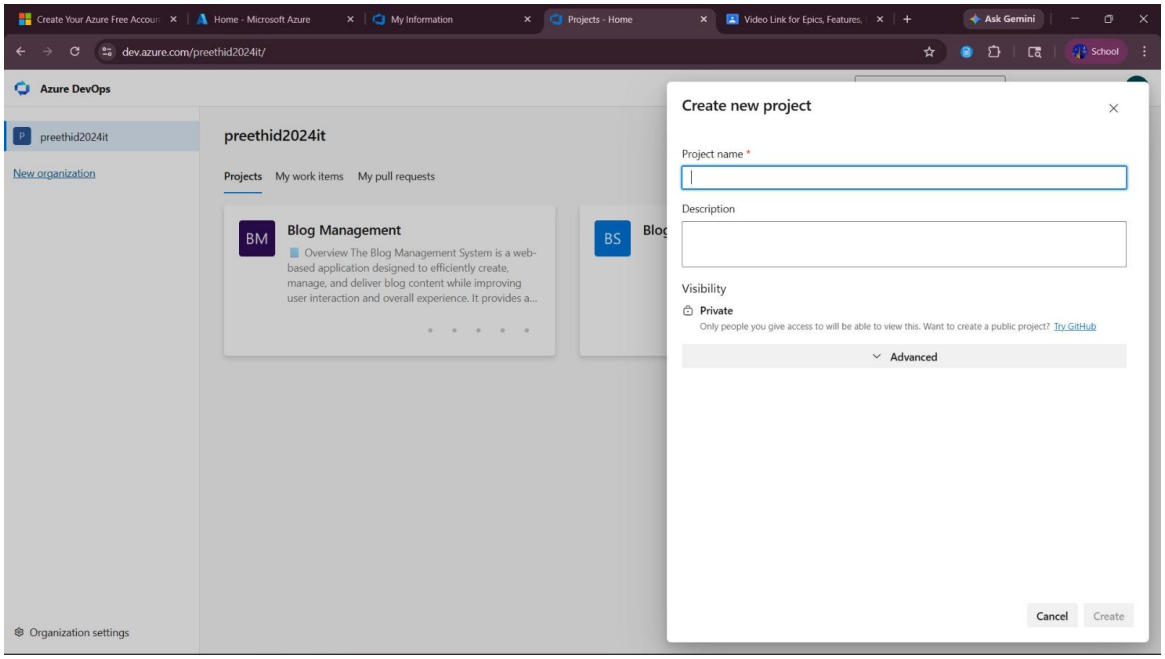
Description:

The Blog Management System is an application that allows users to create, edit, organize, and publish blog posts efficiently. Users can manage categories, upload

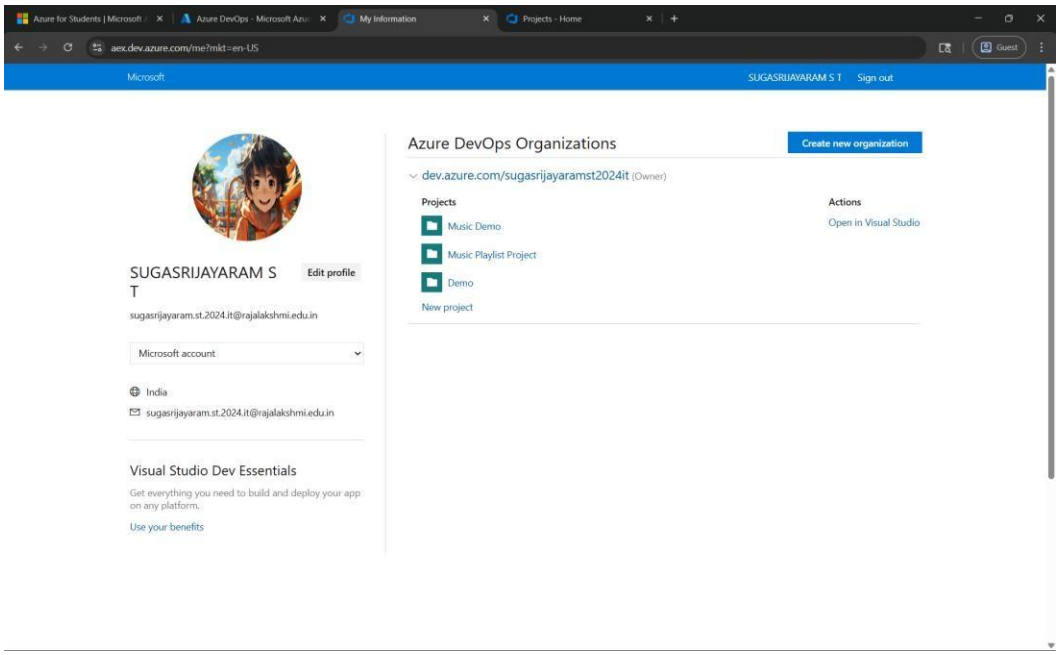
images, schedule posts, and interact through comments. The system also provides user authentication, content management features, and analytics to improve blog performance and reader engagement.

Visibility:
Choose whether you want the project to be **Private** (accessible only to those invited) or **Public** (accessible to anyone).

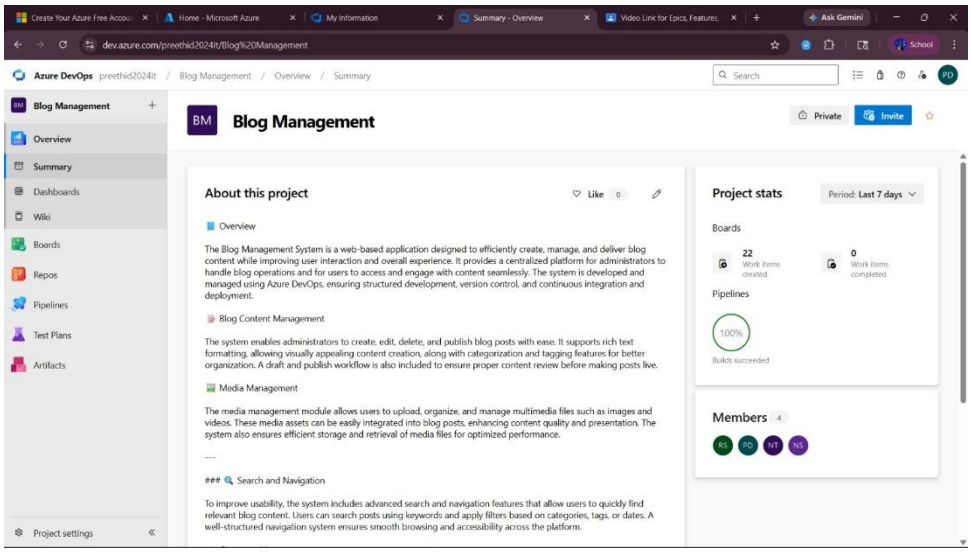
d. Once you've filled out the details, click **Create** to set up your first project.



3.Once logged in, ensure you are in the correct organization. If you're part of multiple organizations, you can switch between them from the top left corner (next to your user profile). Click on the Organization name, and you should be taken to the Azure DevOps Organization Home page.



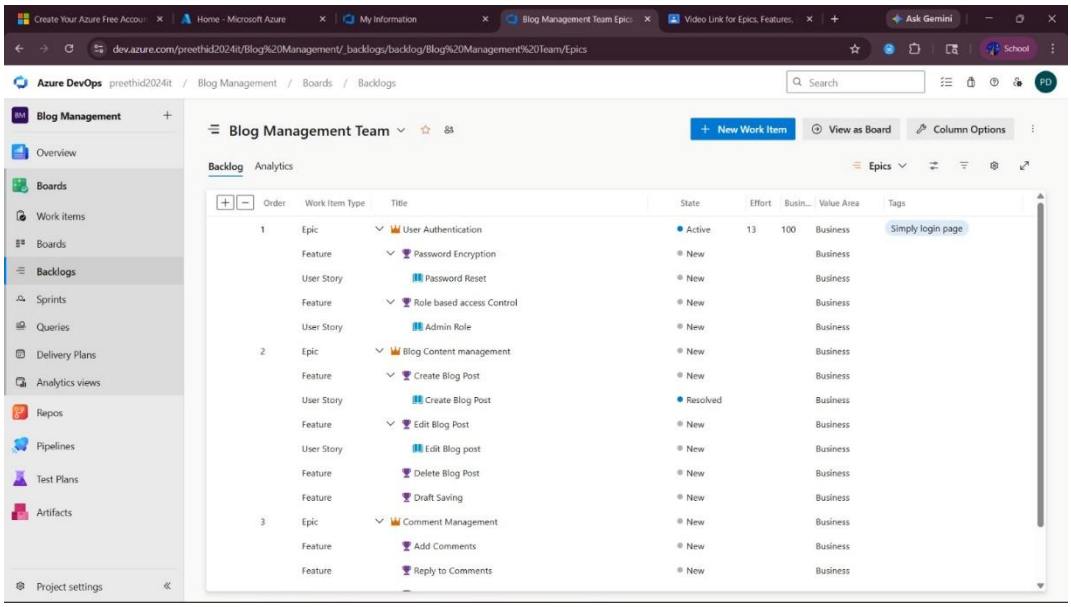
4.Project dashboard



5.To manage user stories:

a. From the **left-hand navigation menu**, click on **Boards**. This will take you to the main **Boards** page, where you can manage work items, backlogs, and sprints.

b. On the **work items** page, you'll see the option to **Add a work item** at the top. Alternatively, you can find a + button or **Add New Work Item** depending on the view you're in. From the **Add a work item** dropdown, select **User Story**. This will open a form to enter details for the new User Story.



Result:

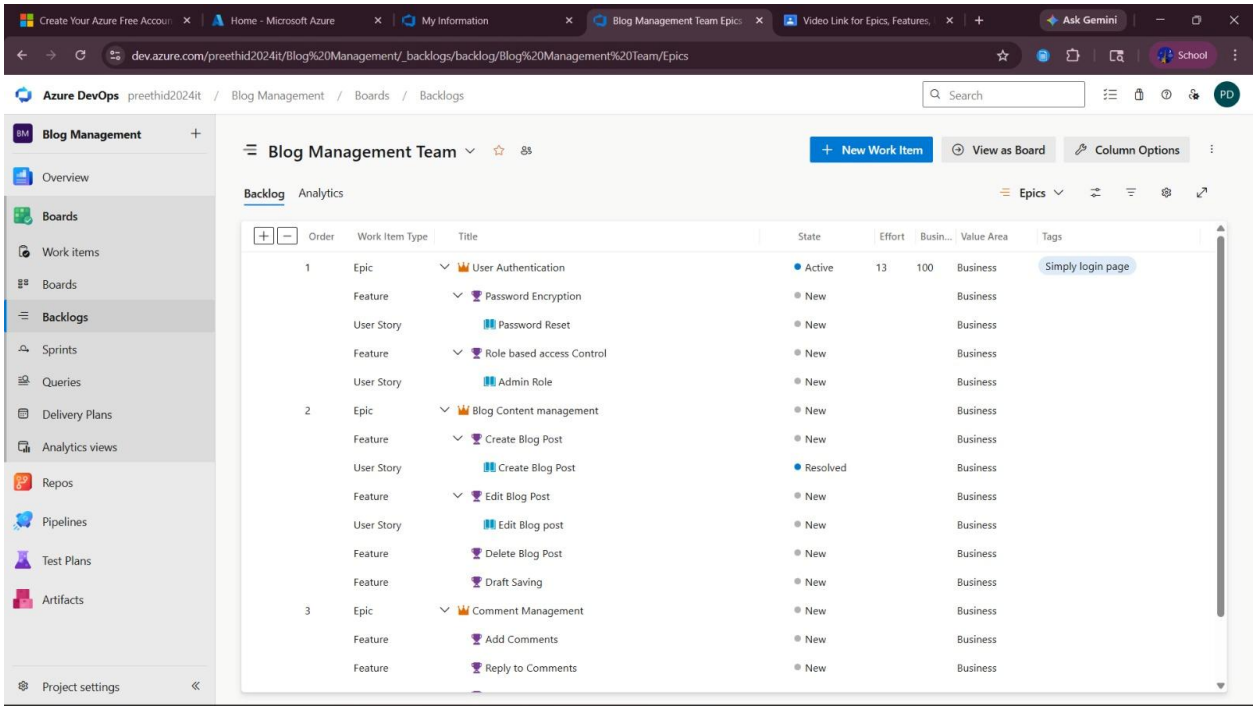
Successfully created an Azure DevOps project with user story management and agile workflow setup.

EXP NO: 3	SETTING UP EPICS, FEATURES, AND USER STORIES FOR PROJECT PLANNING

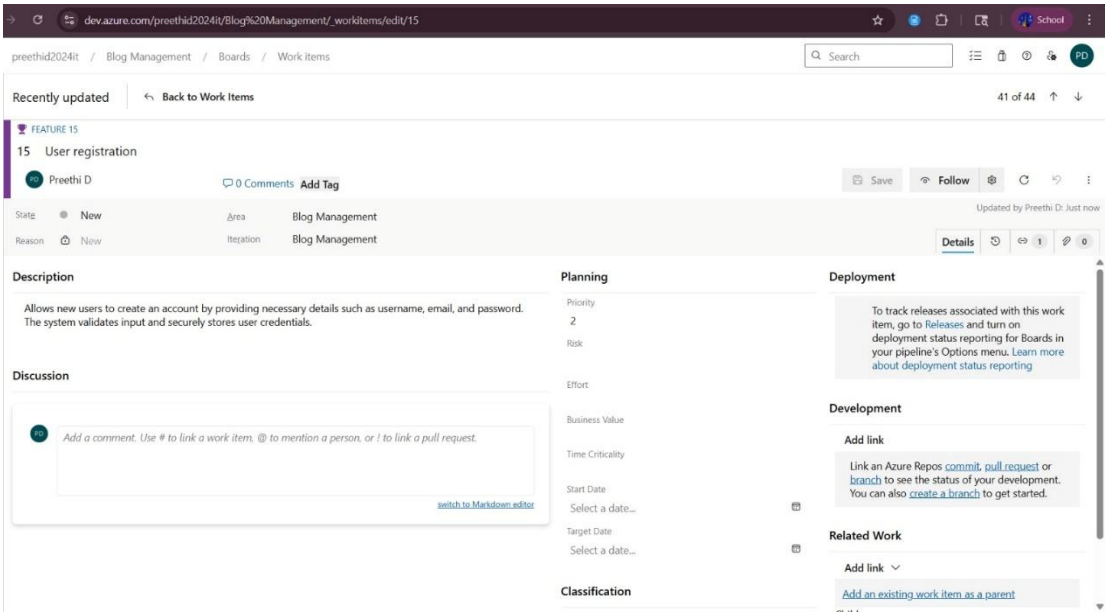
Aim:

To learn about how to create epics, user story, features, backlogs for Blog Management Project.

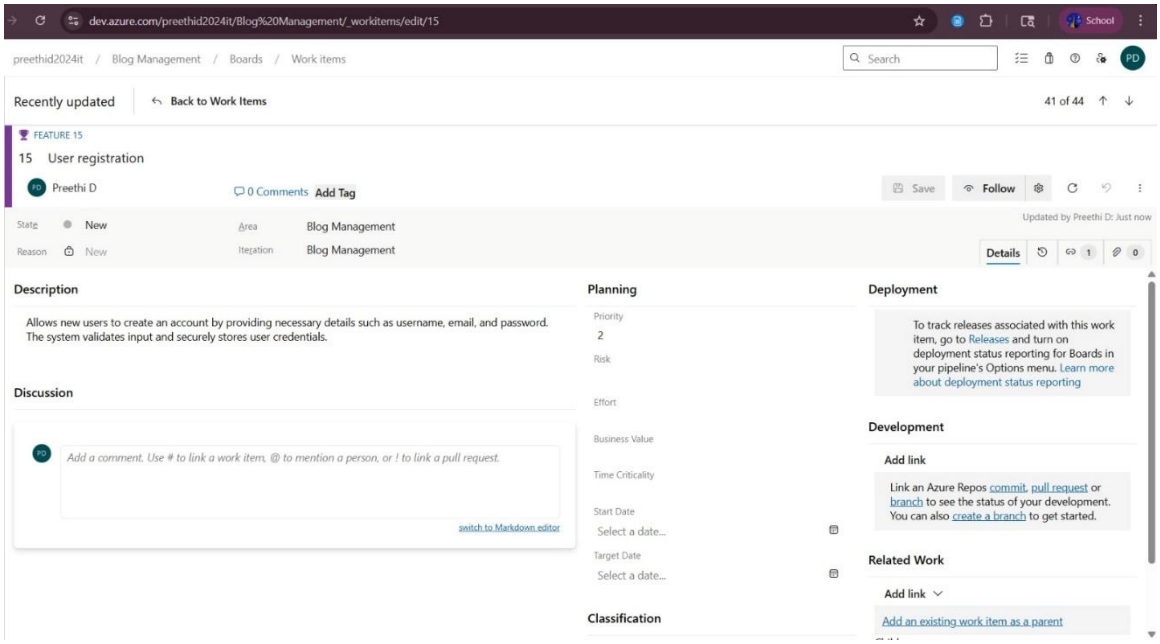
Create Epic, Features, User Stories, Task:



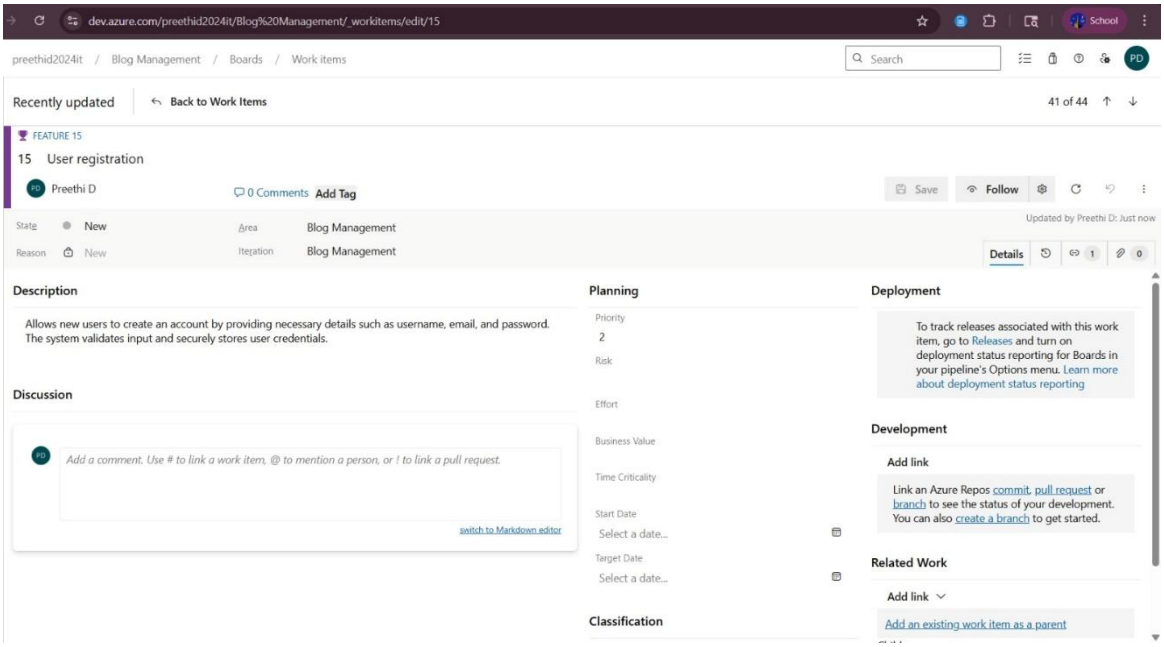
1.Fill in Epics



2.Fill in Features



3.Fill in User Story Details



Result:

Thus, the creation of epics, features, user story and task has been created successfully.

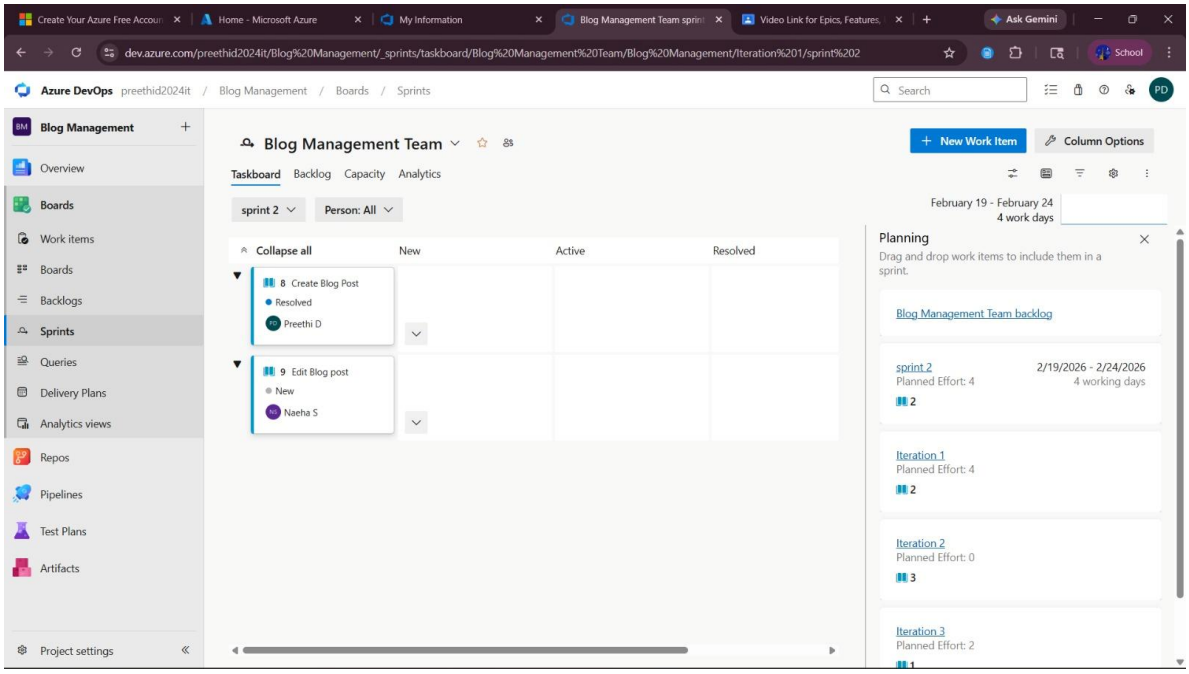
EXP NO: 4	SPRINT PLANNING

Aim:

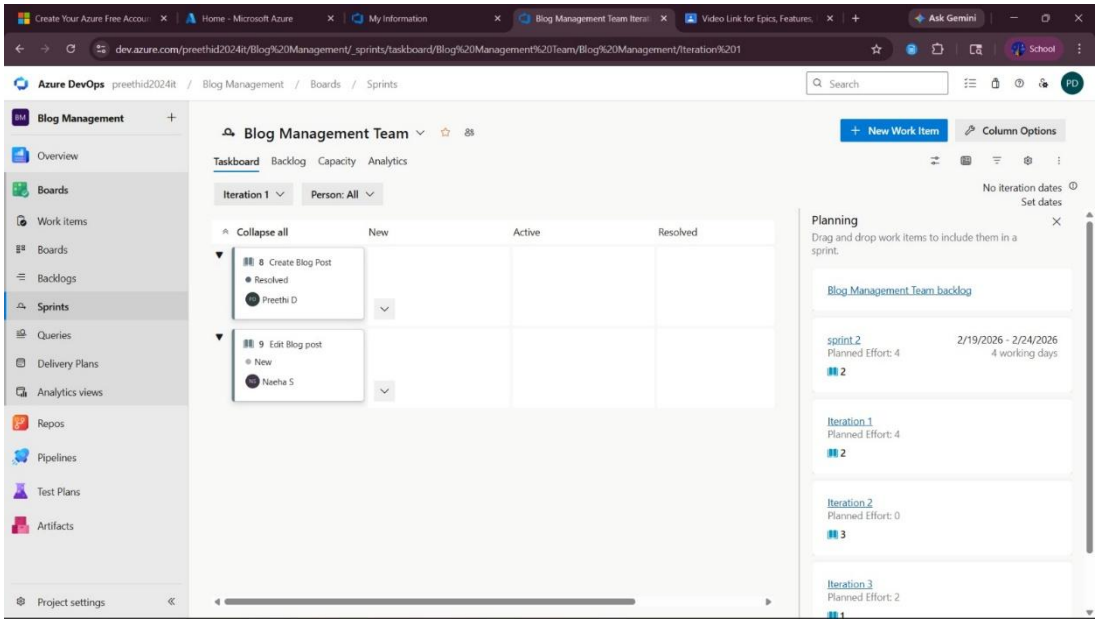
To assign user story to specific sprint for the Blog Management Project.

Sprint Planning:

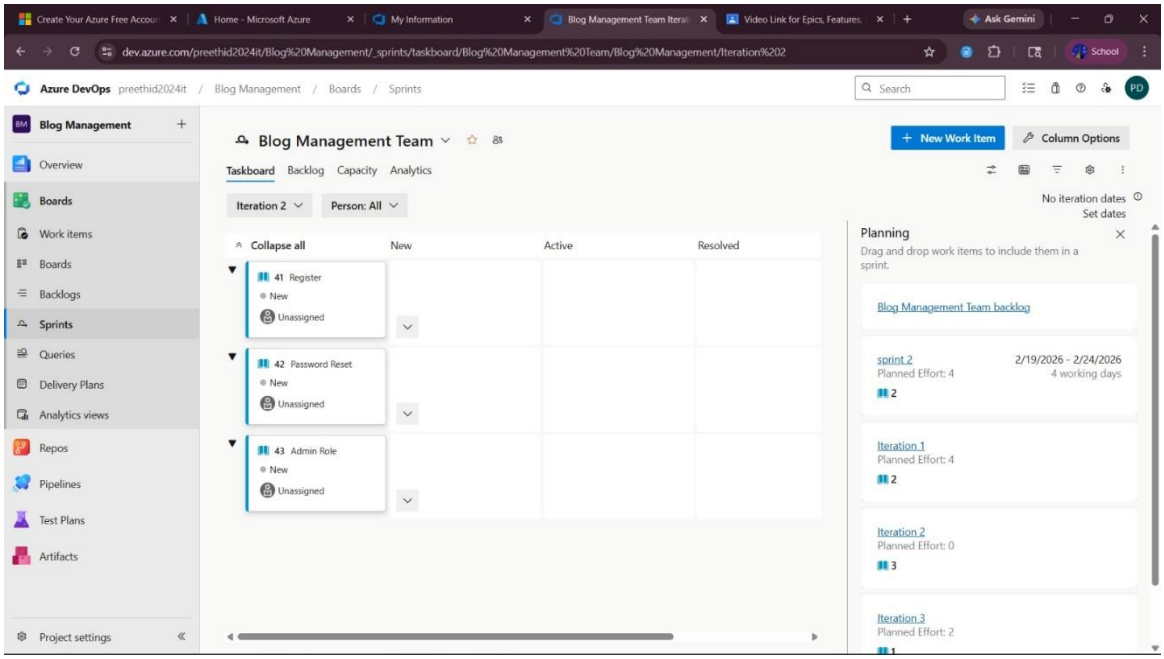
Sprint 1



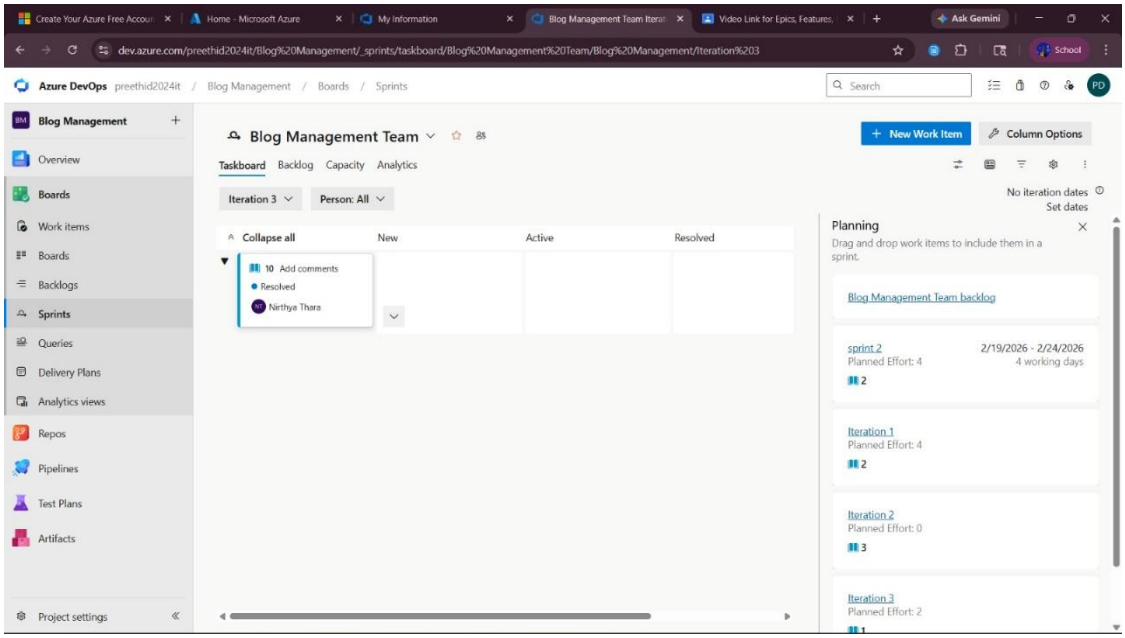
Sprint 2



Sprint 3



Sprint 4



Result:

The Sprints are created for the Blog Management Project.

EXP NO: 5	POKER ESTIMATION

Aim:

Create Poker Estimation for the user stories for Blog Management Project.

Planning Poker Estimation:

User Story ID: 20

Title: User Registration and Authentication

Description:

As a user, I want to register and log in securely so that I can create, manage, and access my personal blog posts and account information.

Story Point Estimation: 8 Points

Team Poker Votes:

Team Member	Vote (Story Points)
Team Member 1	5
Team Member 2	8
Team Member 3	8
Team Member 4	8

Final Consensus: 8 Points

Reason for Estimation:

- Includes multiple features: registration, login, session, logout
- Requires frontend + backend + database integration
- Security implementation (password hashing, authentication)
- Moderate complexity with high risk

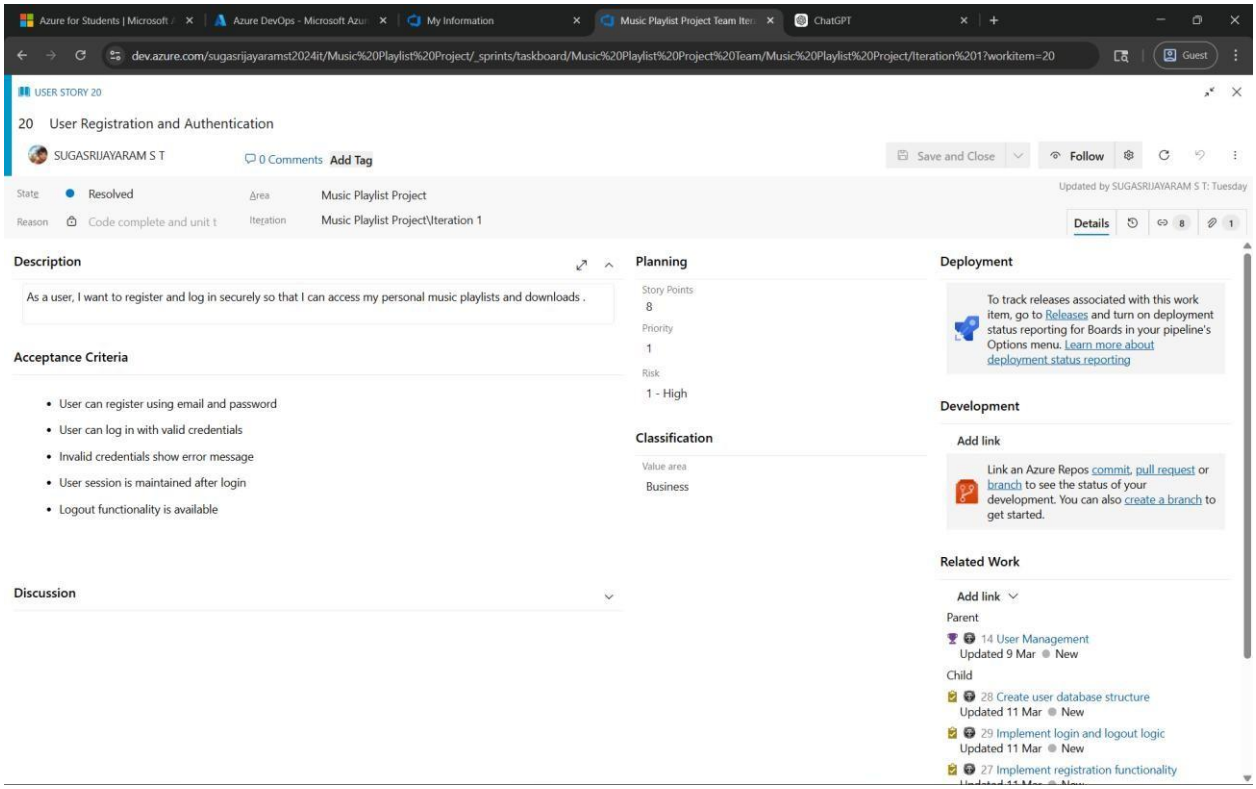
Risks:

- Security vulnerabilities
- Session management issues
- Integration challenges

Acceptance Criteria Covered:

• User registration with email & password
• Login with valid credentials
• Error message for invalid login
• Session maintained after login
• Logout functionality

Conclusion:
Final agreed estimation is 8 story points based on complexity, effort, and risk.



Result:

The Estimation/Story Points is created for the project using Poker Estimation.

EXP NO: 6	DESIGNING CLASS AND SEQUENCE DIAGRAMS FOR PROJECT ARCHITECTURE

Aim:

To Design a Class Diagram and Sequence Diagram for the Blog Management Project.

Class Diagram:

Classes:

- User
- BlogPost
- Category
- Comment
- Tag
- Payment
- Subscription
- Transaction
- Feedback
- Notification
- Advertisement

Key Attributes:

User:

name, email, role, isPremium

BlogPost:

title, content, publishDate, status

Category:

name, description

Comment:

commentText, commentDate

Tag:

tagName

Subscription:

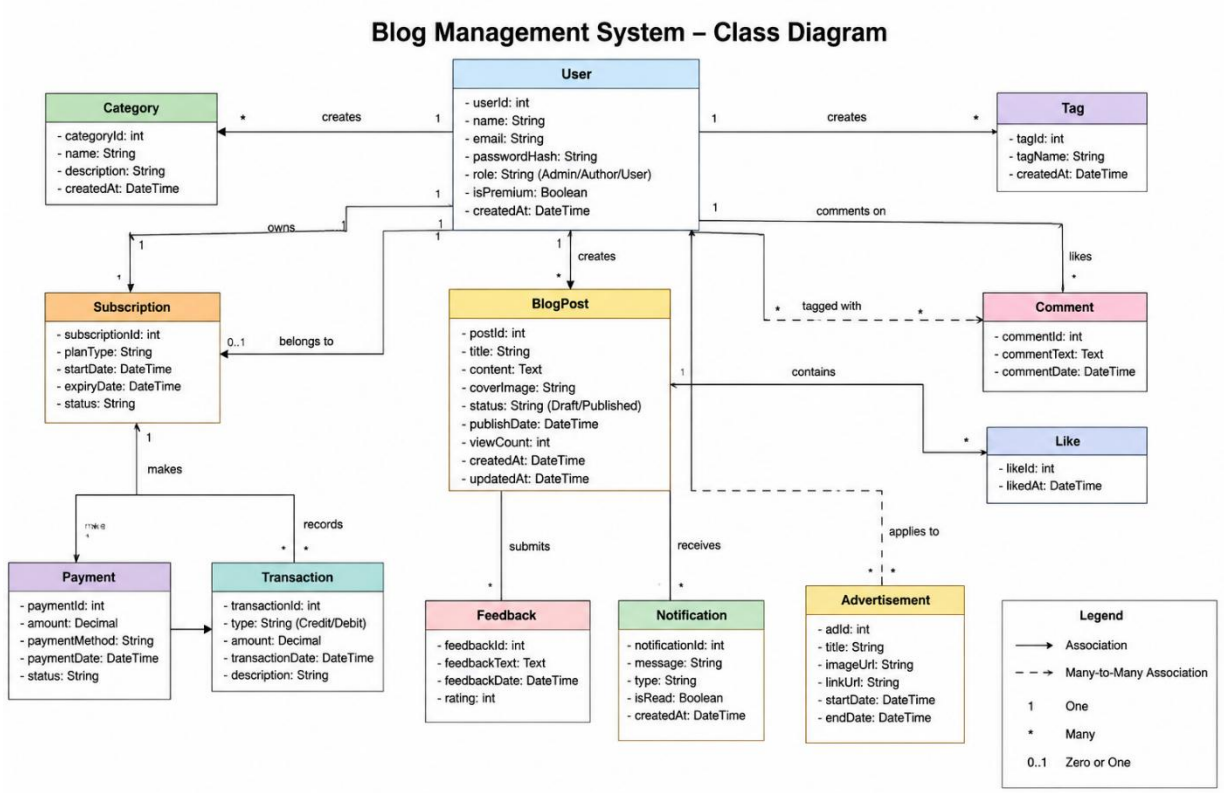
planType, expiryDate

Transaction:

type, amount

Relationships:

• User → creates → BlogPost
• BlogPost → belongs to → Category
• BlogPost → contains → Comment
• BlogPost → tagged with → Tag
• User → comments on → BlogPost
• User → likes → BlogPost
• User → owns → Subscription
• Subscription → records → Transaction
• User → makes → Payment
• User → submits → Feedback
• User → receives → Notification
• Advertisement → applies to → BlogPost/Category



Sequence Diagram:

Actors:

User → Frontend → Backend → Database → Auth Service

Steps:

1. User enters email and password
2. Frontend validates input (email format, password)
3. Frontend sends login request to backend
4. Backend validates request data
5. Backend checks user in database

Condition: User Not Found

• Backend returns error: Invalid credentials

Condition: User Found

6. Backend verifies password
7. If password incorrect → return error
8. If password correct → proceed
9. Backend verifies user with auth service
10. Backend generates JWT token
11. Backend sets session (token expiry)

12. Backend sends success response

---

13. Frontend stores token
14. Frontend redirects to blog dashboard
15. User login successful

Result: The Class Diagram and Sequence Diagram is designed Successfully for the Blog Management Project.

EXP NO: 7	DESIGNING ARCHITECTURAL AND ER DIAGRAMS FOR PROJECT STRUCTURE

Aim:

To Design an Architectural Diagram and ER Diagram for the given Project.

Architectural Diagram:

1. Client Layer

• Browser
• React App
• Redux / Zustand (State Management)

---

2. API Communication Layer

• Axios (HTTP Requests)
• REST API Calls

---

3. Backend Layer

• Express Server
• Routes
• Controllers
• Middleware

---

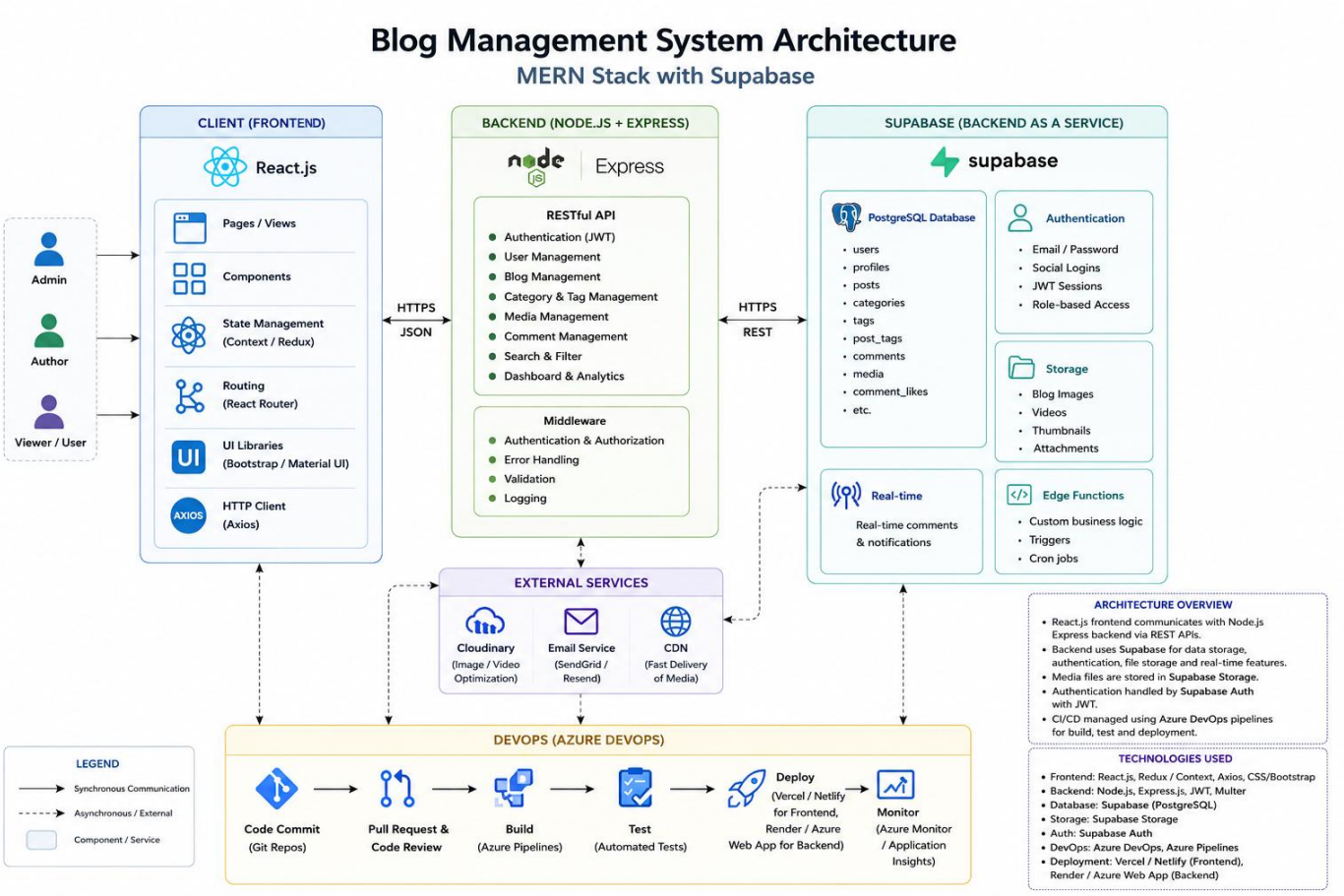
4. Services Layer

• Authentication Service (JWT / Clerk / Firebase Auth)
• Image Upload Service (Cloudinary)
• Notification Service
• Email Service

---

5. Database Layer

• Mongoose Models
• MongoDB



ER Diagram:

Entities & Attributes:

User

- userId
- fullName
- email
- password
- role
- isPremium
- subscriptionPlan
- subscriptionExpiry

BlogPost

• postId
• title
• content
• categoryId
• authorId
• publishDate
• status
• coverImage

Category

• categoryId
• categoryName
• description

Comment

• commentId
• userId
• postId
• commentText
• commentDate

Tag

• tagId
• tagName

Like

• likeId
• userId
• postId

Subscription

• subscriptionId
• userId
• planType

• startDate
• expiryDate
• status

Transaction

• transactionId
• amount
• type
• transactionDate

Payment

• paymentId
• userId
• amount
• status
• paymentMethod

Feedback

• feedbackId
• userId
• message
• rating

Notification

• notificationId
• userId
• message
• type

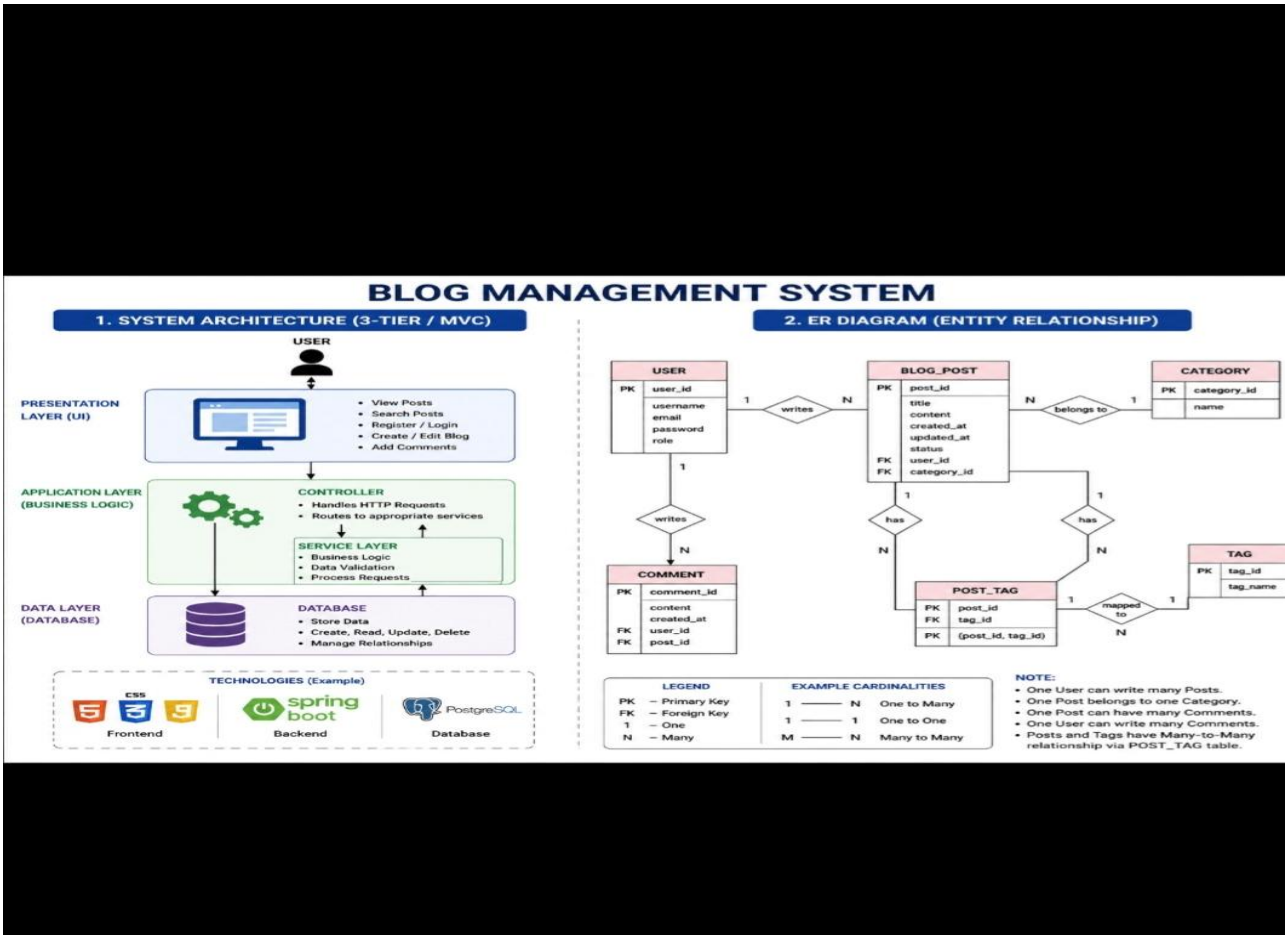
Advertisement

• advertisementId
• title
• content
• targetCategory

• startDate
• endDate

Relationships

• User creates BlogPost (1 : M)
• BlogPost belongs to Category (M : 1)
• BlogPost contains Comment (1 : M)
• BlogPost tagged with Tag (M : N)
• User comments on BlogPost (1 : M)
• User likes BlogPost (M : N)
• User owns Subscription (1 : 1)
• Subscription records Transaction (1 : M)
• User makes Payment (1 : M)
• User submits Feedback (1 : M)
• User receives Notification (1 : M)
• Advertisement applies to BlogPost / Category
• Tag connected to BlogPost (M : N)



Result:

The Architecture Diagram and ER Diagram is designed Successfully for the Blog Management Project.

EXP NO: 8	TESTING – TEST PLANS AND TEST CASES

Aim:

Test Plans and Test Case and write two test cases for at least two user stories showcasing the happy path and error scenarios in azure DevOps platform.

Test Planning and Test Case:

Test Case Design Procedure

1. Understand Core Features of the Application

o User Signup & Login
o Creating and Managing Blog Posts
o Viewing Categories and Tags
o Editing and Publishing Blog Posts
o Commenting and Liking Blog Posts
o Managing User Profiles and Subscriptions

---

2. Define User Interactions

o Each test case simulates a real user behaviour (e.g., logging in, creating a blog post, adding a comment).

---

3. Design Happy Path Test Cases

o Focused on validating that all features function as expected under normal conditions.
o Example: User logs in successfully, creates a blog post, publishes content, or comments on a post.

---

4. Design Error Path Test Cases

o Simulate negative or unexpected scenarios to test robustness and error handling.
o Example: Login fails with invalid credentials, blog post save fails due to network issue, empty comment submission.

---

5. Break Down Steps and Expected Results

o Each test case contains step-by-step actions and a corresponding expected outcome.
o Ensures clarity for both testers and automation scripts.

---

6. Use Clear Naming and IDs

o Test cases are named clearly (e.g., TC01 – Successful Signup, TC06 – Create and Publish Blog Post).
o Helps in quick identification and linking to user stories or features.

---

7. Separate Test Suites

o Grouped test cases based on functionality (e.g., Login, Blog Management, Comments, Notifications).

o Improves organization and test execution flow in Azure DevOps.

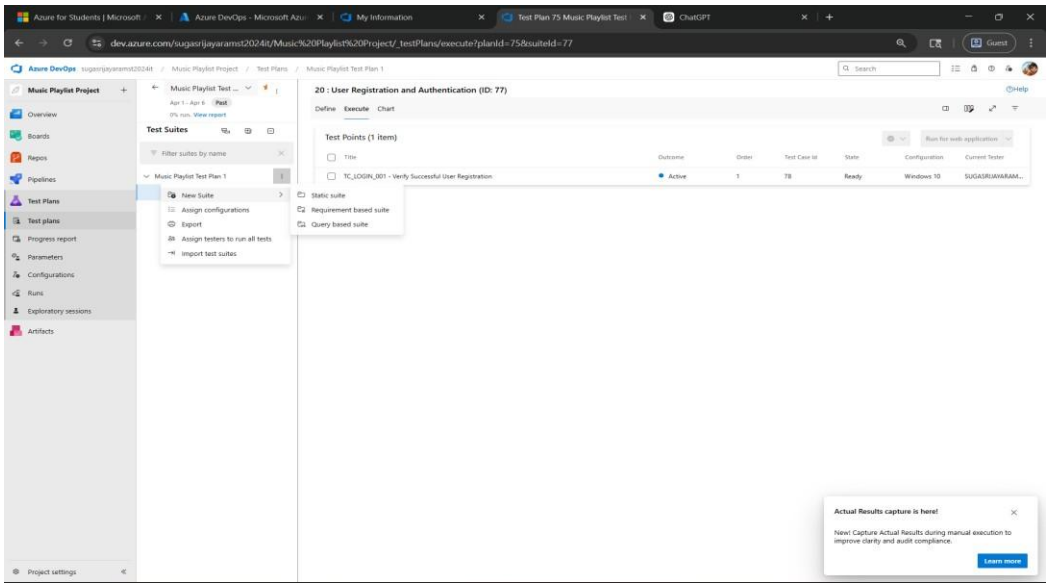
---

8. Prioritize and Review

o Critical user actions are marked high-priority.

o Reviewed for completeness and traceability against feature requirements.

1.New test plan



2.Test case

USER STORIES

● As a user, I want to register and log in securely so that I can access and manage my blog account. (ID: 20)

● As a user, I want to create and manage blog posts so that I can publish and organize my content easily. (ID: 23)

Test Suites

Test Suite: TS01 - User Registration and Authentication (ID: 20)

1. TC01 – User Registration Successfully w/o Error

• Action:

o Go to the Sign-Up page
o Enter valid name, email, and password
o Click "Sign Up"

• Expected Results:

o Sign-Up form is displayed
o Fields accept values without error
o Account is created successfully
o User is redirected to dashboard

• Type: Happy Path

---

2. TC02 – User Registration Failed with Invalid User Details

• Action:

o Go to the Sign-Up page
o Enter invalid details (empty fields / wrong email format / weak password)
o Click "Sign Up"

• Expected Results:

o Validation errors are displayed
o Account is not created

• Type: Error Path

---

3. TC03 – User Login Successfully with Valid Credentials

• Action:

o Go to Login page
o Enter valid email and password
o Click "Login"

• Expected Results:

o Login form is displayed
o Credentials are accepted
o User is redirected to dashboard

• Type: Happy Path

---

4. TC04 – Login with Invalid Credentials

• Action:

o Go to Login page
o Enter invalid email or password
o Click "Login"

• Expected Results:

o Input is accepted
o Error message "Invalid username or password" is shown

• Type: Error Path

Test Suite: TS02 - Blog Creation and Management (ID: 23)

1. TC05 – Create Blog Post Successfully

• Action:

o Login with valid credentials
o Navigate to "My Blogs"
o Click "Create Blog"
o Enter blog title and content
o Click "Publish/Save"

• Expected Results:

o Blog post is created successfully
o Blog appears in the blog list

• Type: Happy Path

---

2. TC06 – Edit Blog Post Successfully

• Action:

o Open an existing blog post
o Click "Edit"
o Update title or content
o Click "Save Changes"

• Expected Results:

o Blog content is updated successfully
o Updated changes are visible in the blog post

• Type: Happy Path

---

3. TC07 – Create Blog Post with Empty Fields

• Action:

o Navigate to "Create Blog" page
o Leave title/content empty
o Click "Publish/Save"

• Expected Results:

o Validation errors are displayed
o Blog post is not created

• Type: Error Path

---

4. TC08 – Delete Blog Post Successfully

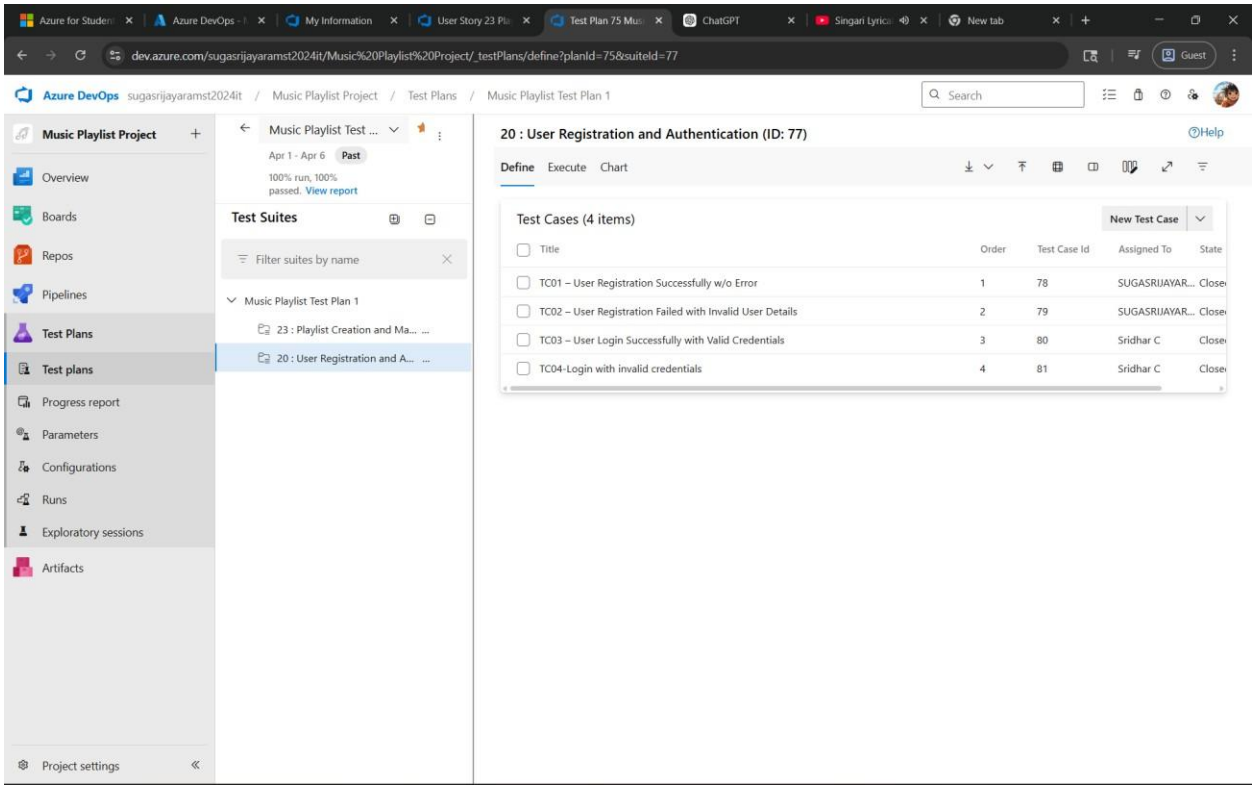
• Action:

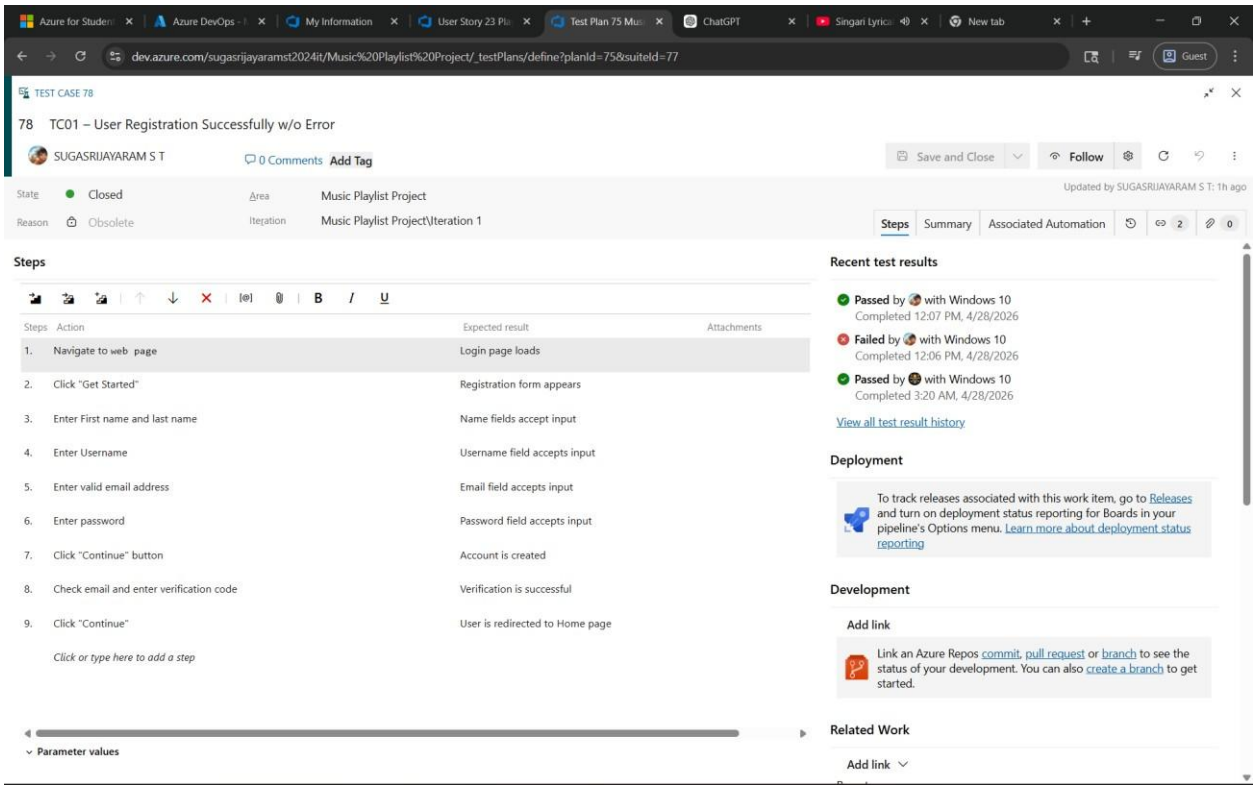
o Navigate to "My Blogs"
o Click "Delete" on a blog post
o Confirm deletion

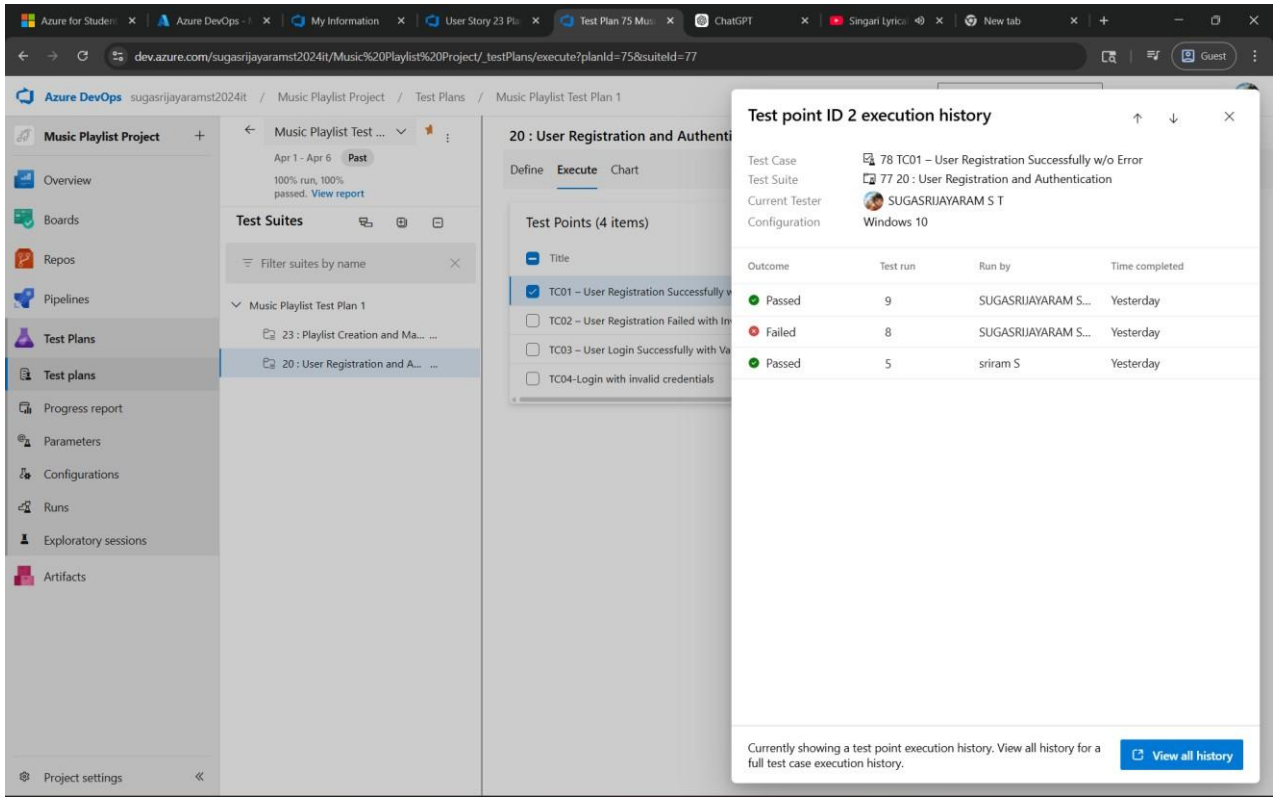
• Expected Results:

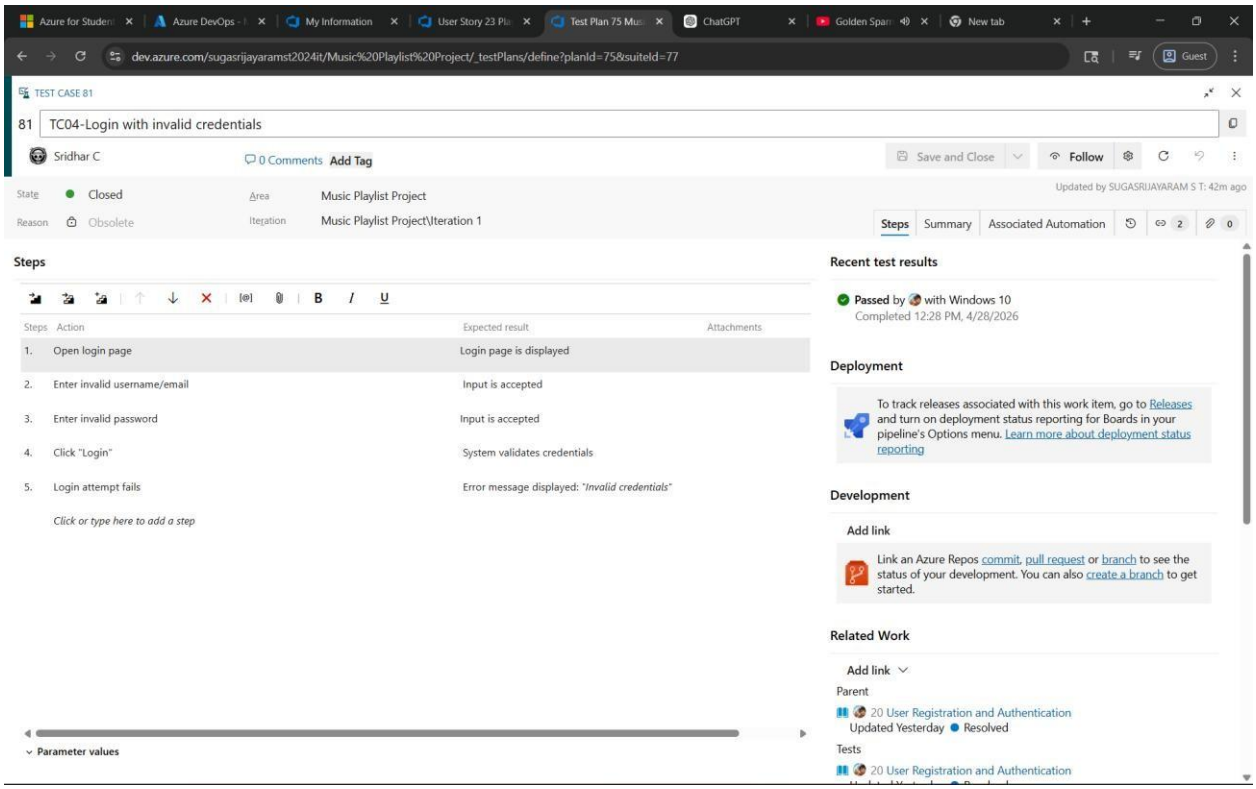
o Blog post is deleted successfully
o Blog is removed from the list
o Changes are saved

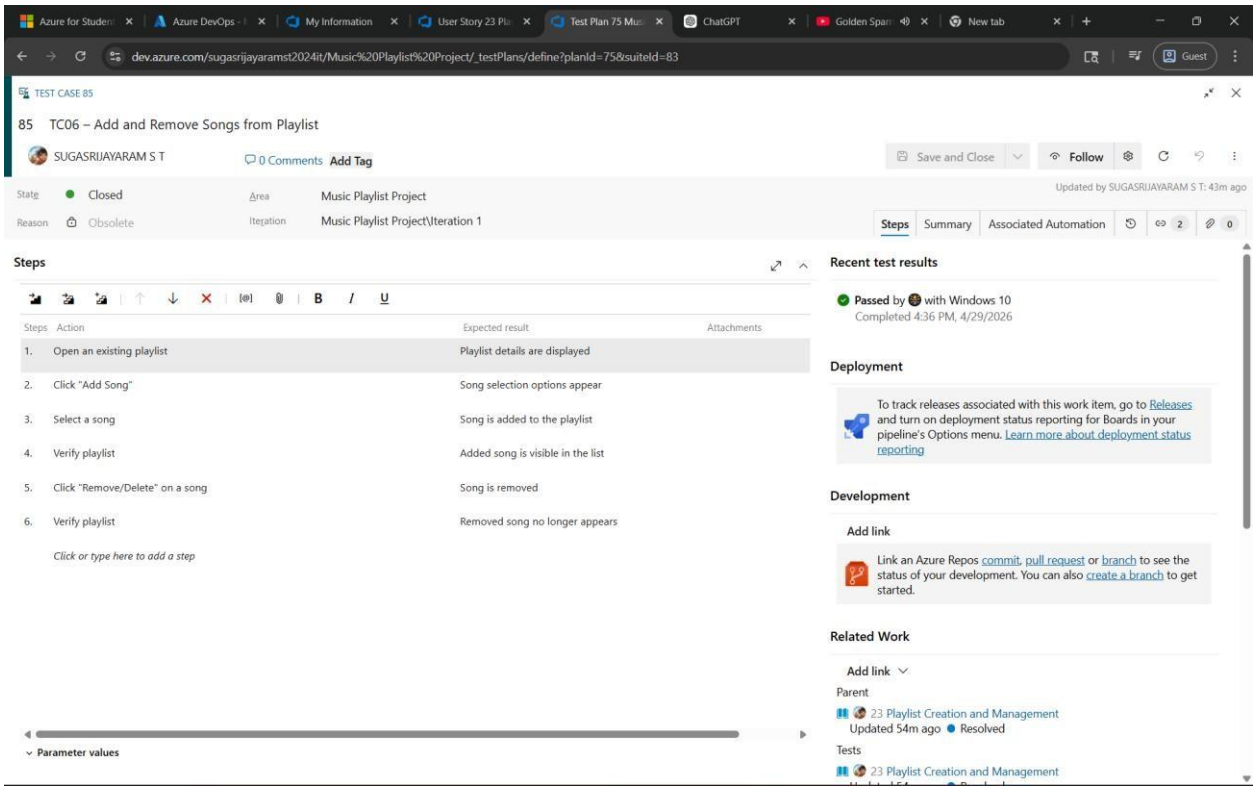
• Type: Happy Path



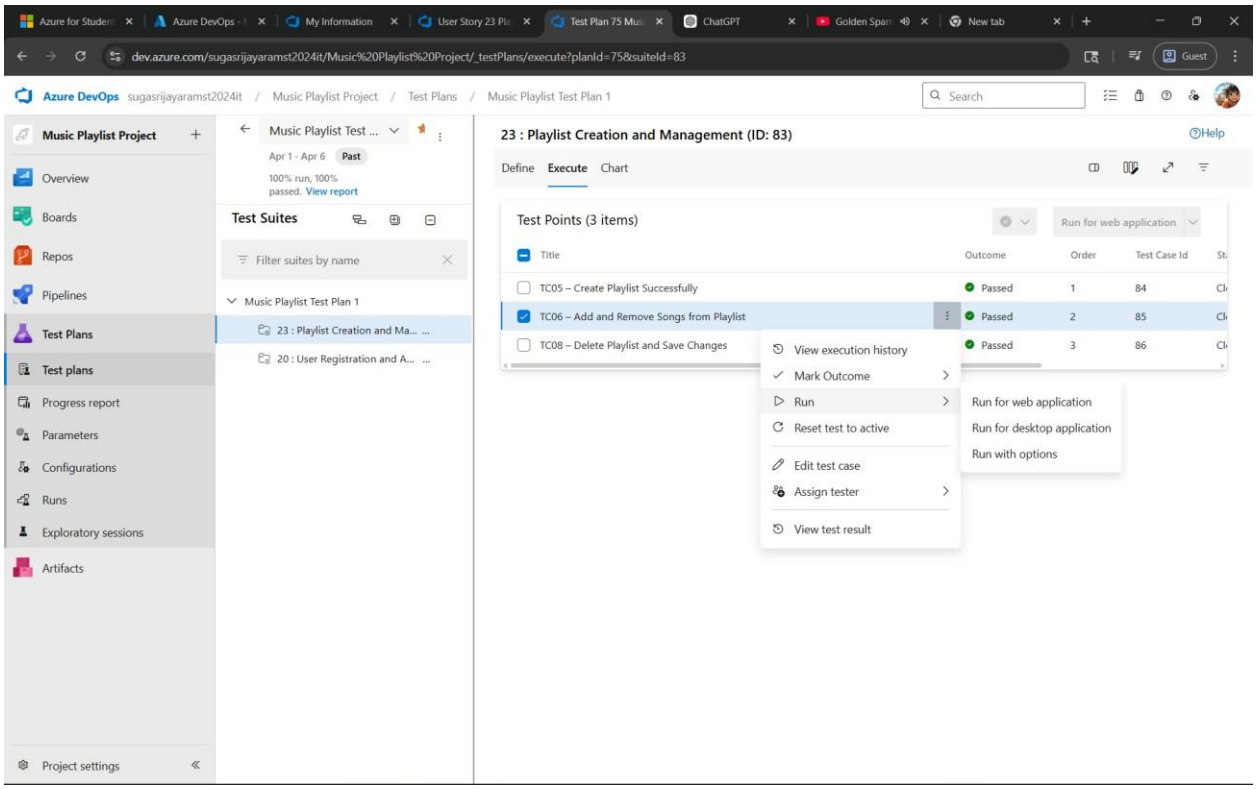

TEST CASE 78
78 TC01 – User Registration Successfully w/o Error
SUGASRIJAYARAM S T
0 Comments Add Tag
Save and Close
Follow
State Closed
Reason Obsolete
Area Music Playlist Project
Iteration Music Playlist Project\Iteration 1
Updated by SUGASRIJAYARAM S T: 1h ago
Steps Summary Associated Automation
Steps
Steps Action Expected result Attachments
1. Navigate to web page Login page loads
2. Click "Get Started" Registration form appears
3. Enter First name and last name Name fields accept input
4. Enter Username Username field accepts input
5. Enter valid email address Email field accepts input
6. Enter password Password field accepts input
7. Click "Continue" button Account is created
8. Check email and enter verification code Verification is successful
9. Click "Continue" User is redirected to Home page
Click or type here to add a step
Parameter values
Recent test results
Passed by with Windows 10
Completed 12:07 PM, 4/28/2026
Failed by with Windows 10
Completed 12:06 PM, 4/28/2026
Passed by with Windows 10
Completed 3:20 AM, 4/28/2026
View all test result history
Deployment
To track releases associated with this work item, go to Releases and turn on deployment status reporting for Boards in your pipeline's Options menu. Learn more about deployment status reporting
Development
Add link
Link an Azure Repos commit, pull request or branch to see the status of your development. You can also create a branch to get started.
Related Work
Add link

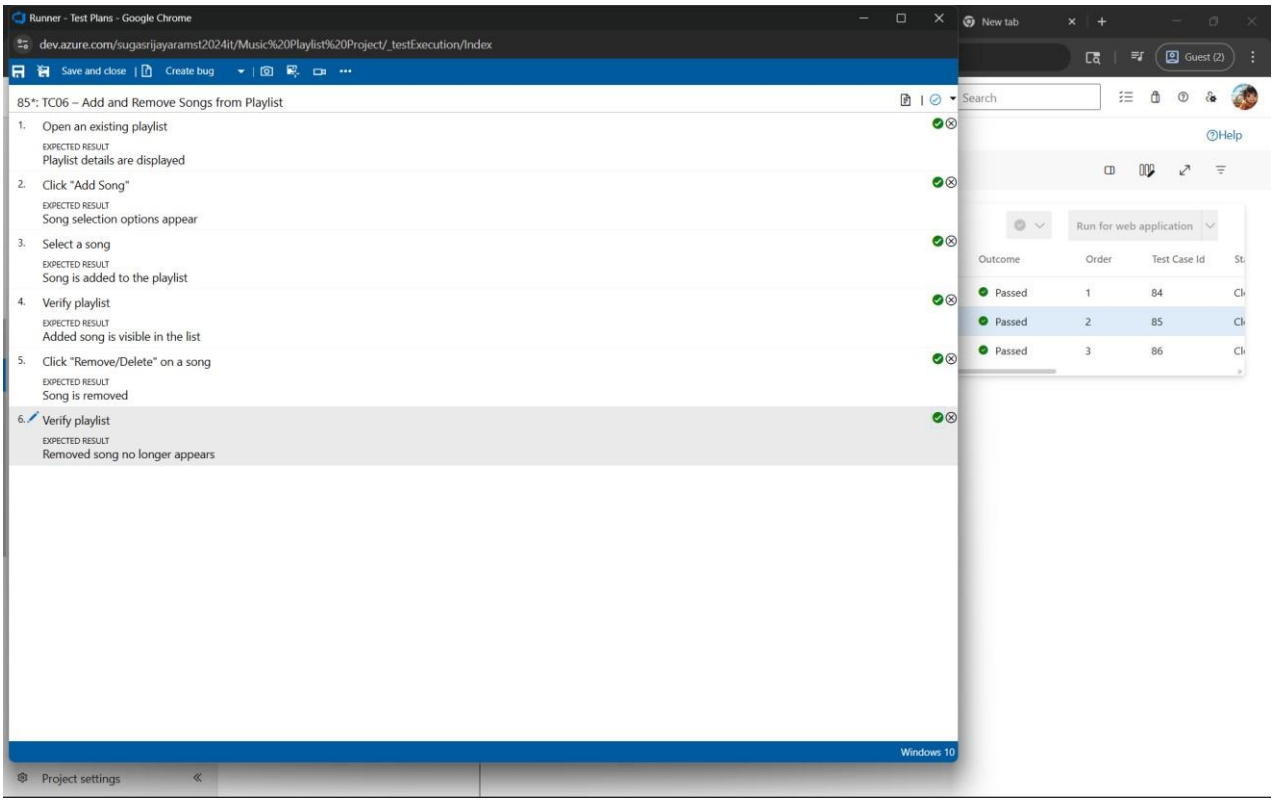

Azure DevOps sugasrijayaramst2024it / Music Playlist Project / Test Plans / Music Playlist Test Plan 1
Music Playlist Project
Overview
Boards
Repos
Pipelines
Test Plans
Test plans
Progress report
Parameters
Configurations
Runs
Exploratory sessions
Artifacts
Project settings
Music Playlist Test ...
Apr 1 - Apr 6 Past
100% run, 100% passed. View report
Test Suites
Filter suites by name
Music Playlist Test Plan 1
23 : Playlist Creation and Ma...
20 : User Registration and A...
20 : User Registration and Authenti
Define Execute Chart
Test Points (4 items)
Title
TC01 – User Registration Successfully
TC02 – User Registration Failed with In
TC03 – User Login Successfully with Va
TC04-Login with invalid credentials
Test point ID 2 execution history
Test Case 78 TC01 – User Registration Successfully w/o Error
Test Suite 77 20 : User Registration and Authentication
Current Tester SUGASRIJAYARAM S T
Configuration Windows 10
Outcome Test run Run by Time completed
Passed 9 SUGASRIJAYARAM S... Yesterday
Failed 8 SUGASRIJAYARAM S... Yesterday
Passed 5 sriram S Yesterday
Currently showing a test point execution history. View all history for a full test case execution history.
View all history



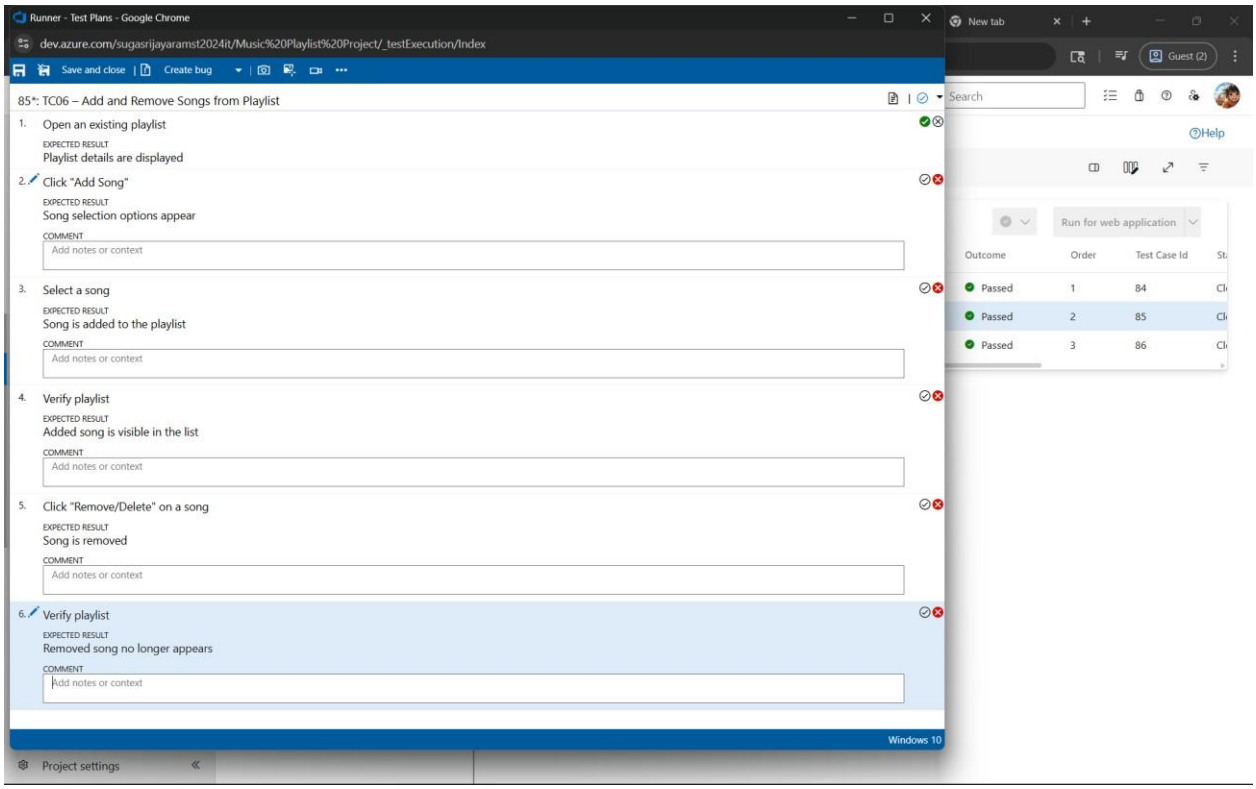


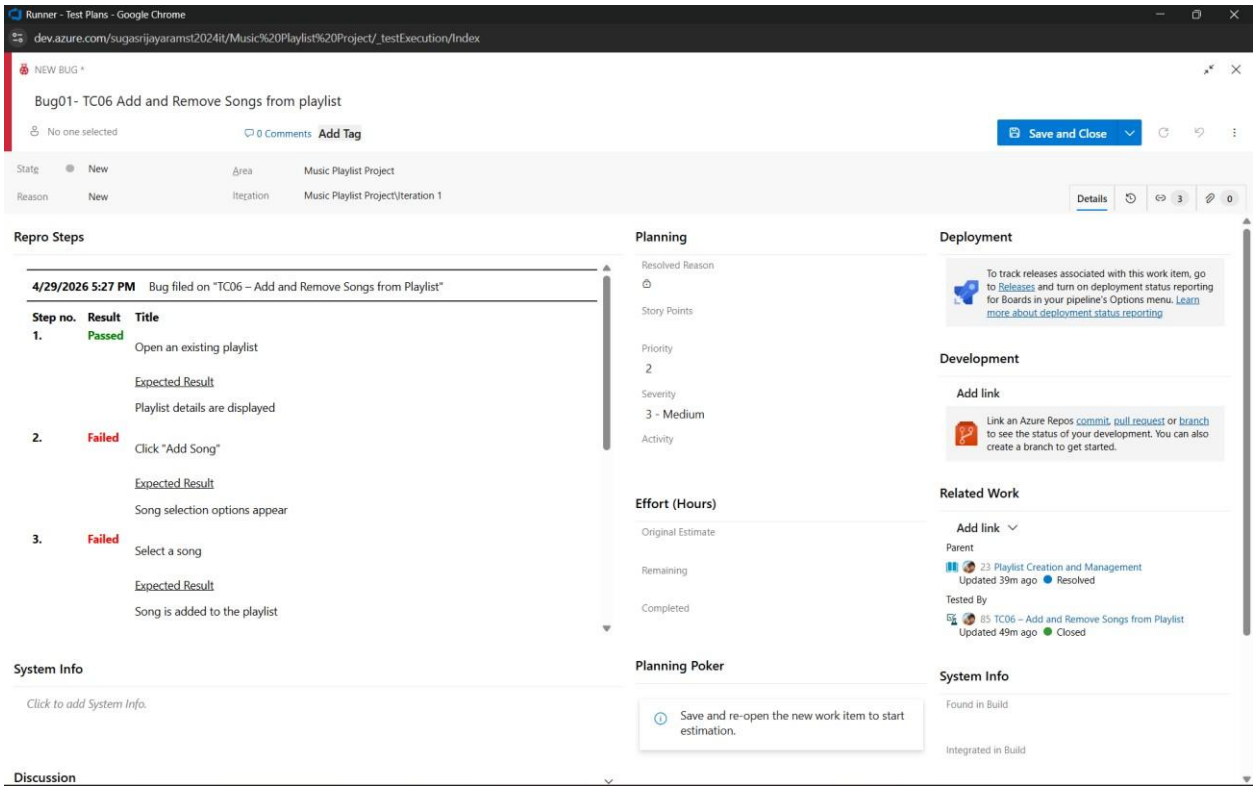
4.Running the test cases



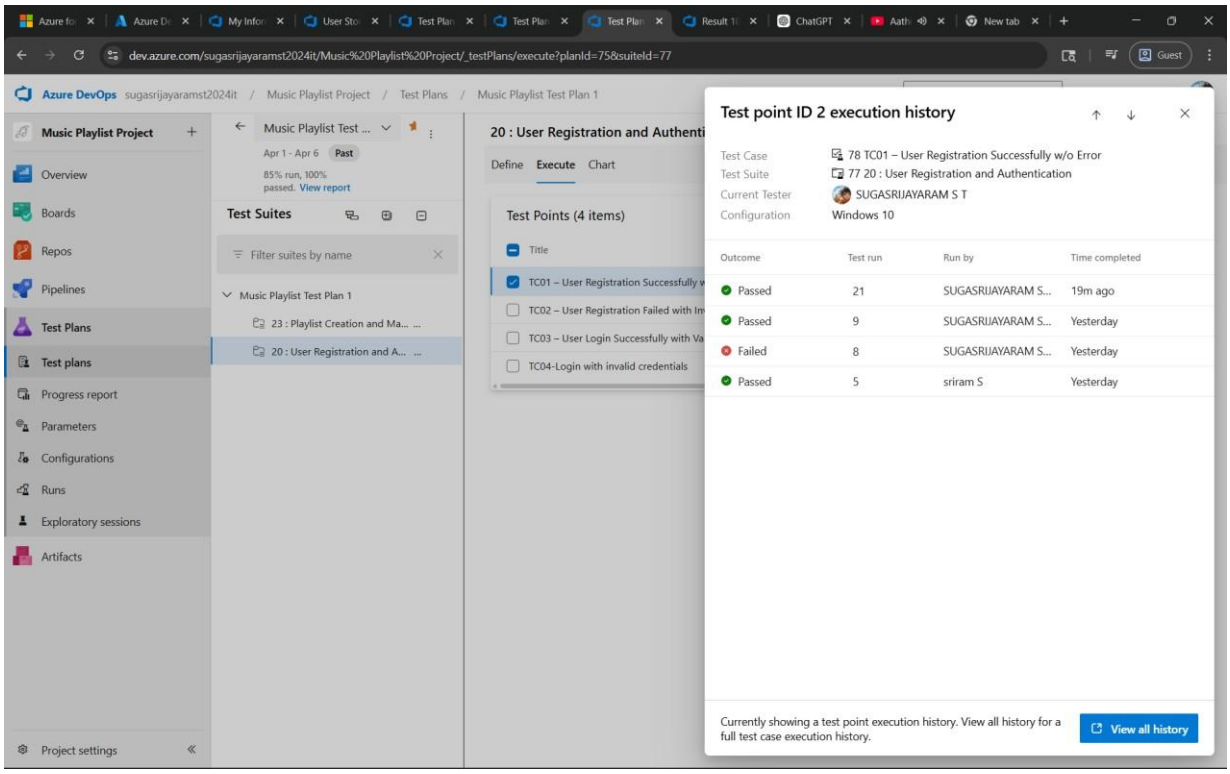


5.Creating the bug



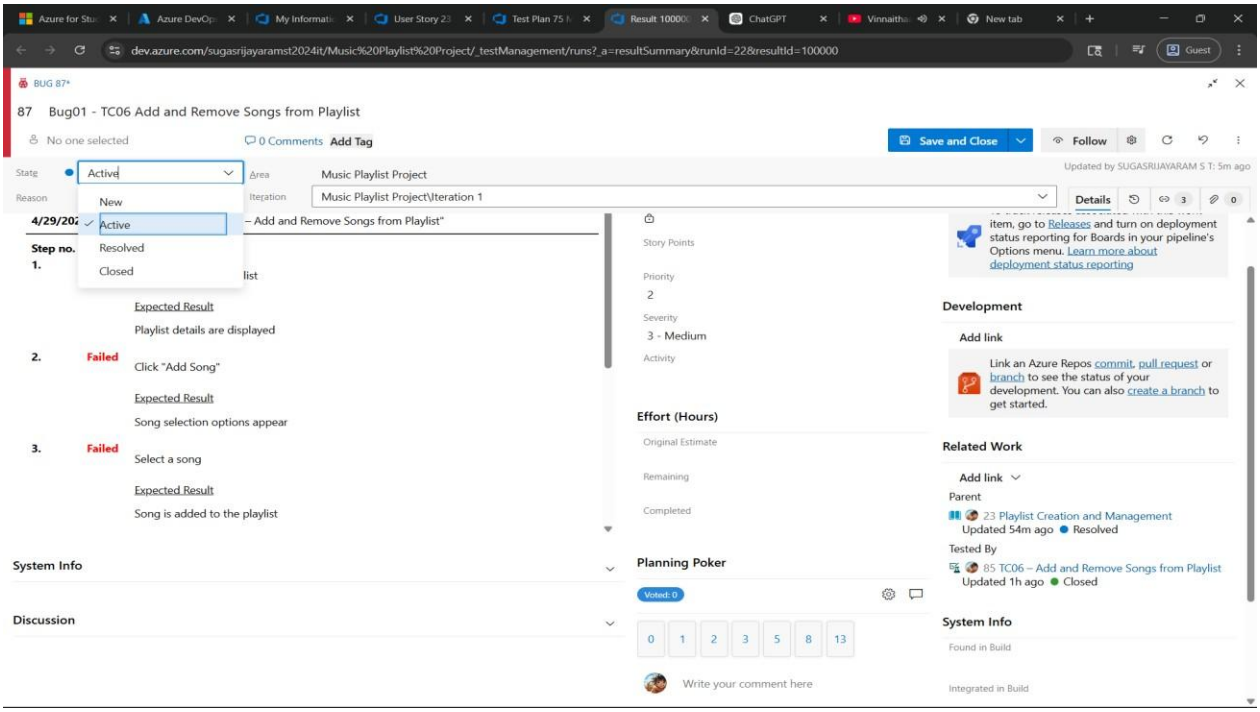


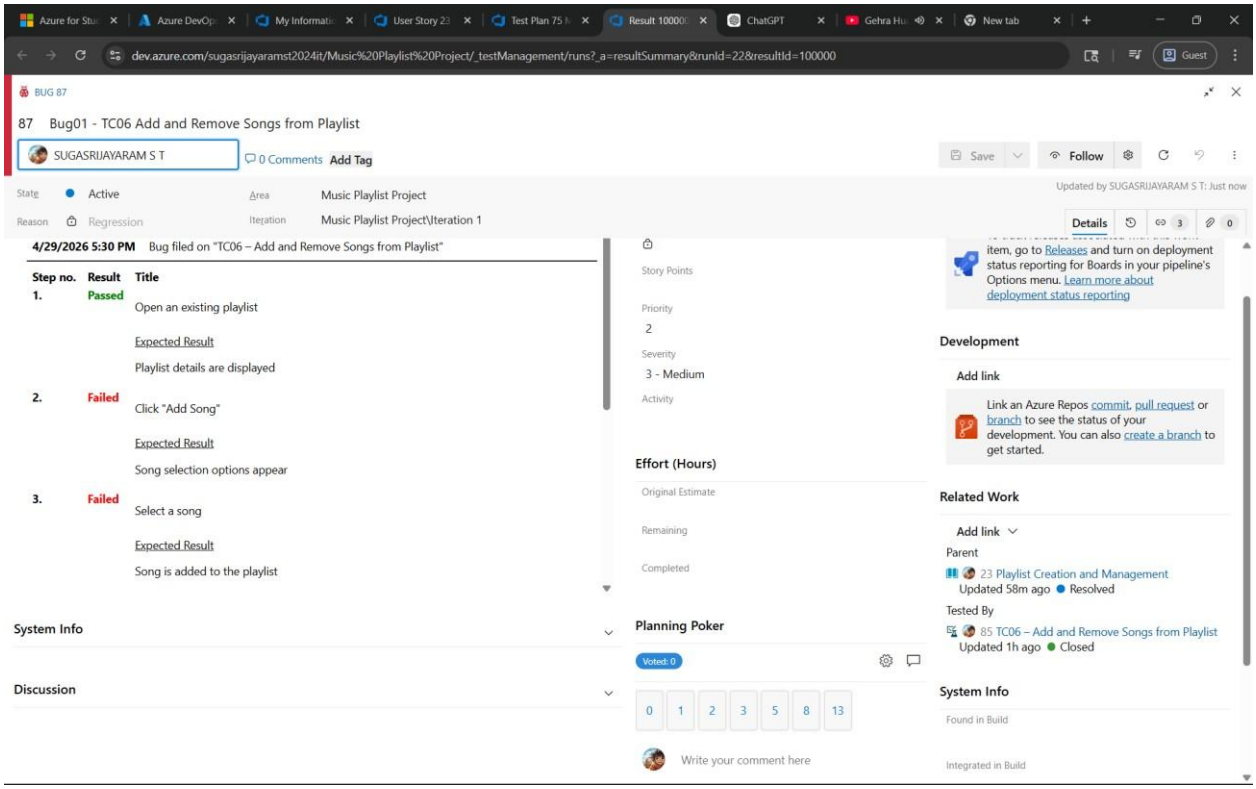
6.Test case results



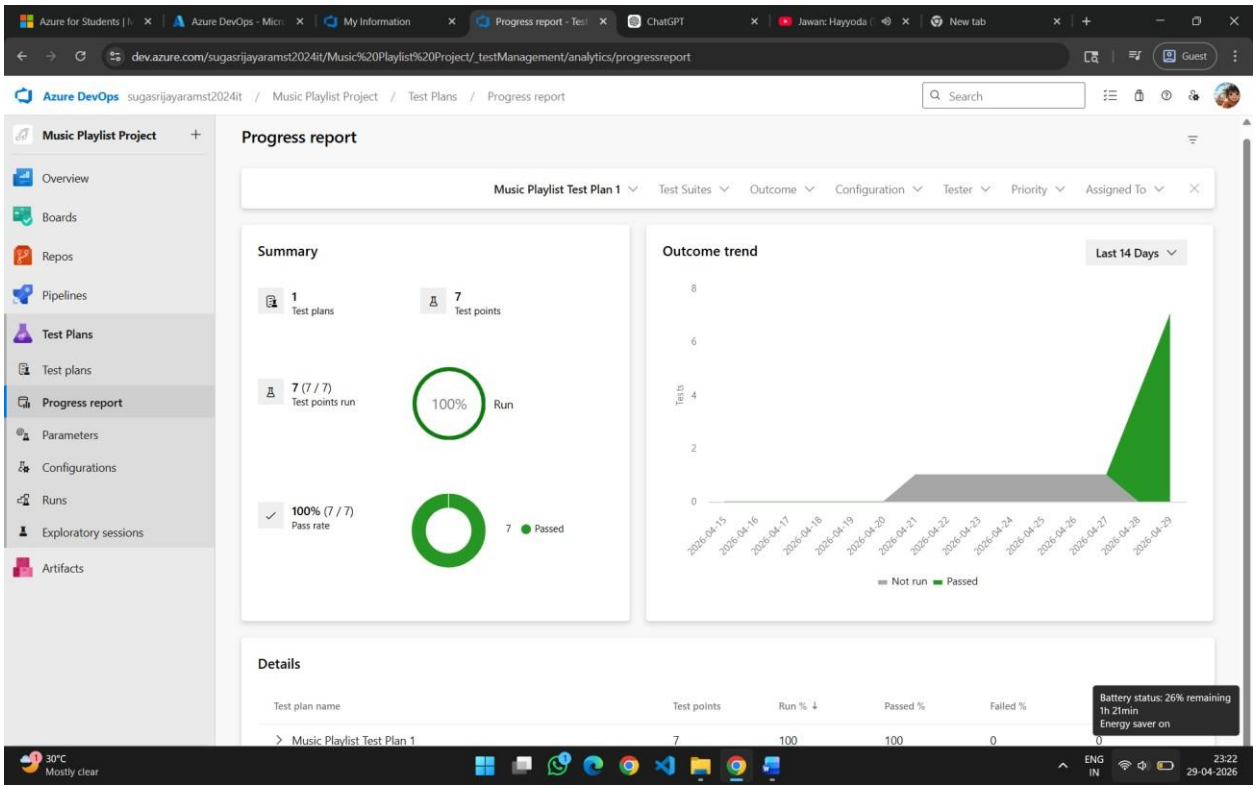
7.Bug report

- Assigning bug to the developer and changing state to Active





8.Progress report



Result:

The test plans and test cases for the user stories is created in Azure DevOps with Happy Path and Error Path.

EXP NO: 9	LOAD TESTING AND PIPELINES

Aim:

To create an Azure Load Testing resource and run a load test to evaluate the performance of a target endpoint and to create and demonstrate an Azure DevOps pipeline for automating application builds, tests, and deployment.

Load Testing

Azure Load Testing:

Azure Load Testing allows you to simulate high traffic and stress tests for your web applications and APIs to understand how they perform under load. It helps identify performance bottlenecks, scalability issues, and optimize resource usage before deployment.

Steps to Create an Azure Load Testing Resource:

Before you run your first test, you need to create the Azure Load Testing resource:

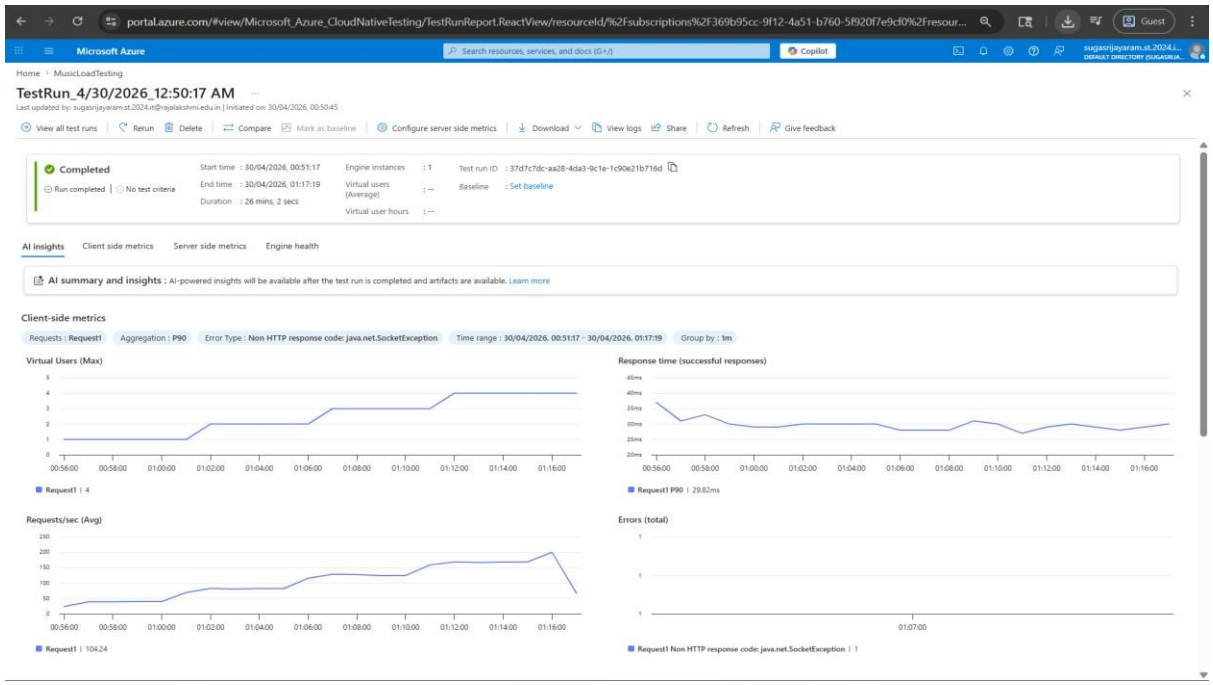
1. Sign in to Azure Portal
   Go to https://portal.azure.com and log in.
2. Create the Resource ○ Go to *Create a resource* → Search for "Azure Load Testing".
   - Select Azure Load Testing and click Create.
3. Fill in the Configuration Details ○ Subscription*:* Choose your Azure subscription. ○ *Resource Group:* Create new or select an existing one. ○ *Name:* Provide a unique name (no special characters).
   - *Location:* Choose the region for hosting the resource.
4. (Optional) Configure tags for categorization and billing.
5. Click Review + Create, then Create.
6. Once deployment is complete, click Go to resource.

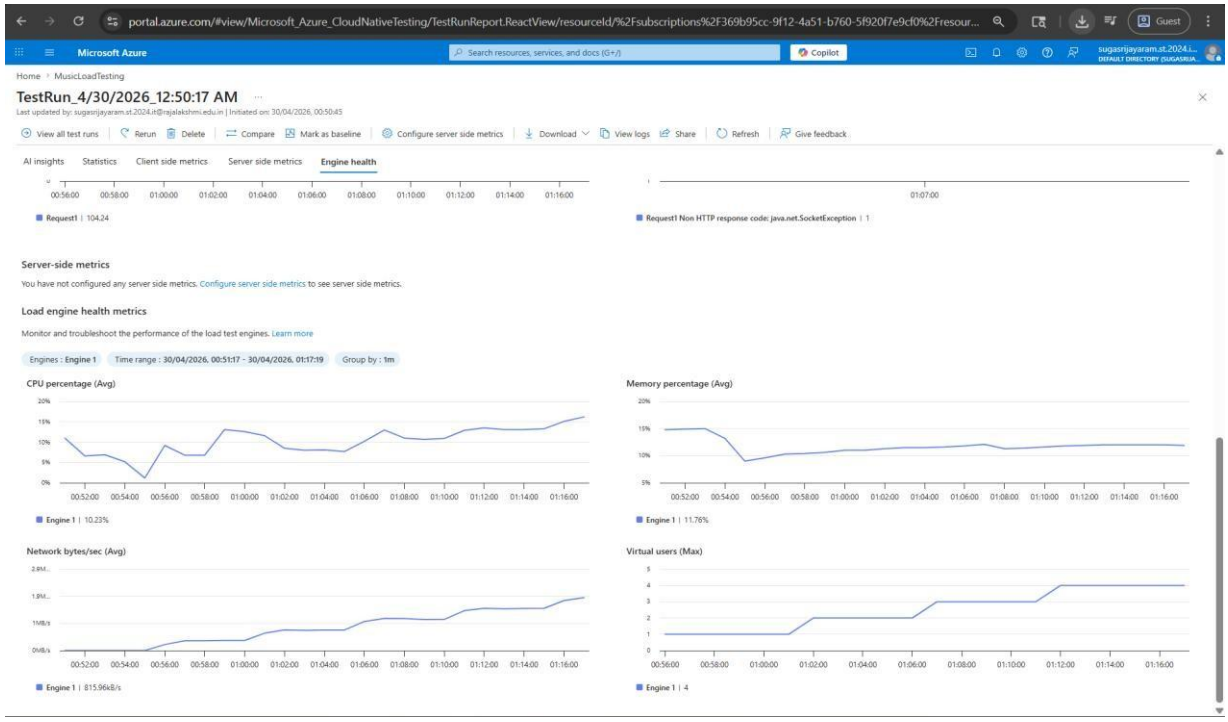
Steps to Create and Run a Load Test:

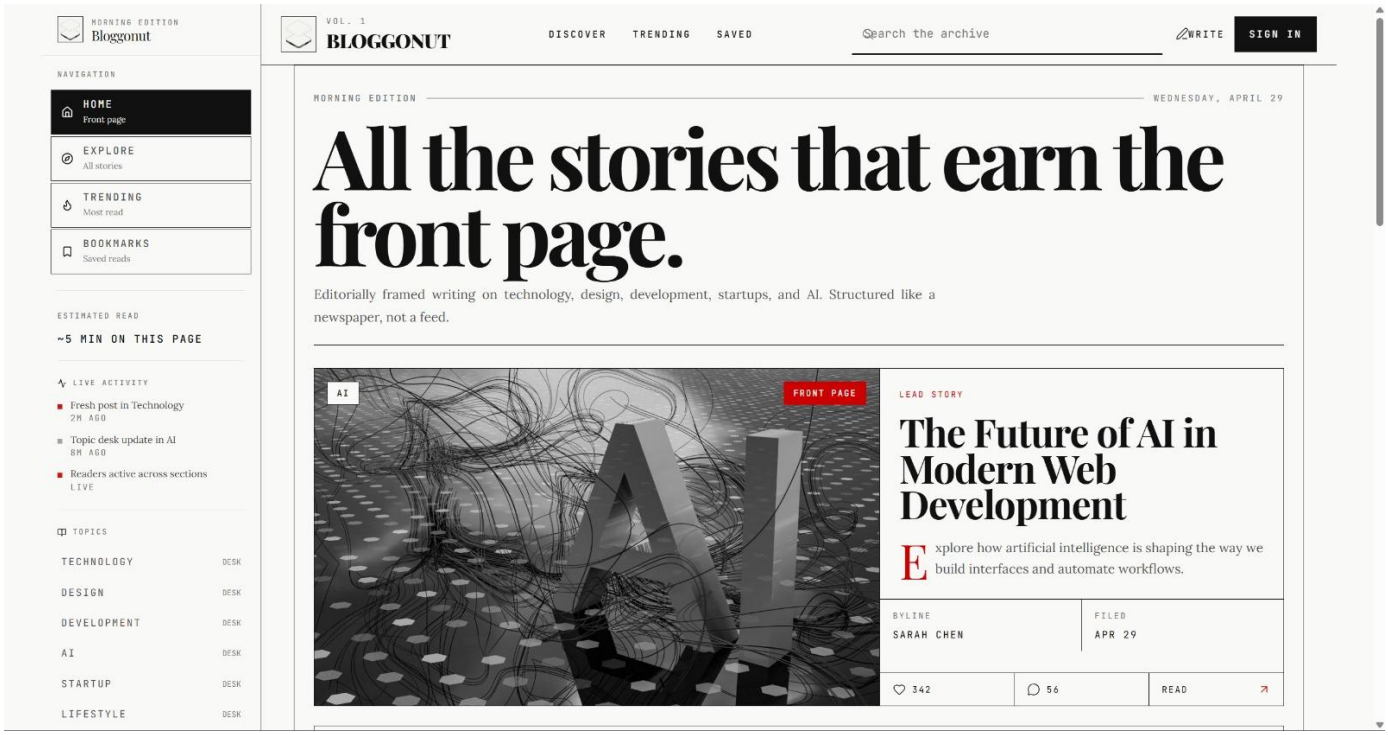
Once your resource is ready:

1. Go to your Azure Load Testing resource and click Add HTTP requests > Create.
2. Basics Tab ○ *Test Name:* Provide a unique name.
   - *Description:* (Optional) Add test purpose.
   - *Run After Creation:* Keep checked.
3. Load Settings ○ *Test URL:* Enter the target endpoint (e.g., https://yourapi.com/products).
4. Click Review + Create → Create to start the test.

Load Testing







Pipelines

Description:

This experiment demonstrates how to connect a GitHub-hosted Flask-based music recommendation project with Azure DevOps. The pipeline will automatically install dependencies, run basic tests, and publish artifacts. This ensures that every commit triggers checks for reliability and smooth deployment.

Steps:

1. Connect GitHub to Azure DevOps:
    - In Azure DevOps, create a new project.
    - Create a pipeline and select GitHub as the source.
    - Authorize access to your GitHub repository, ensuring that Azure DevOps can pull the repository for your pipeline.
2. Create azure-pipelines.yml in Your Repo Root:
    - In your GitHub repository, create a new file called azure-pipelines.yml in the root directory.
    - Add the following basic pipeline configuration for Python and Flask:

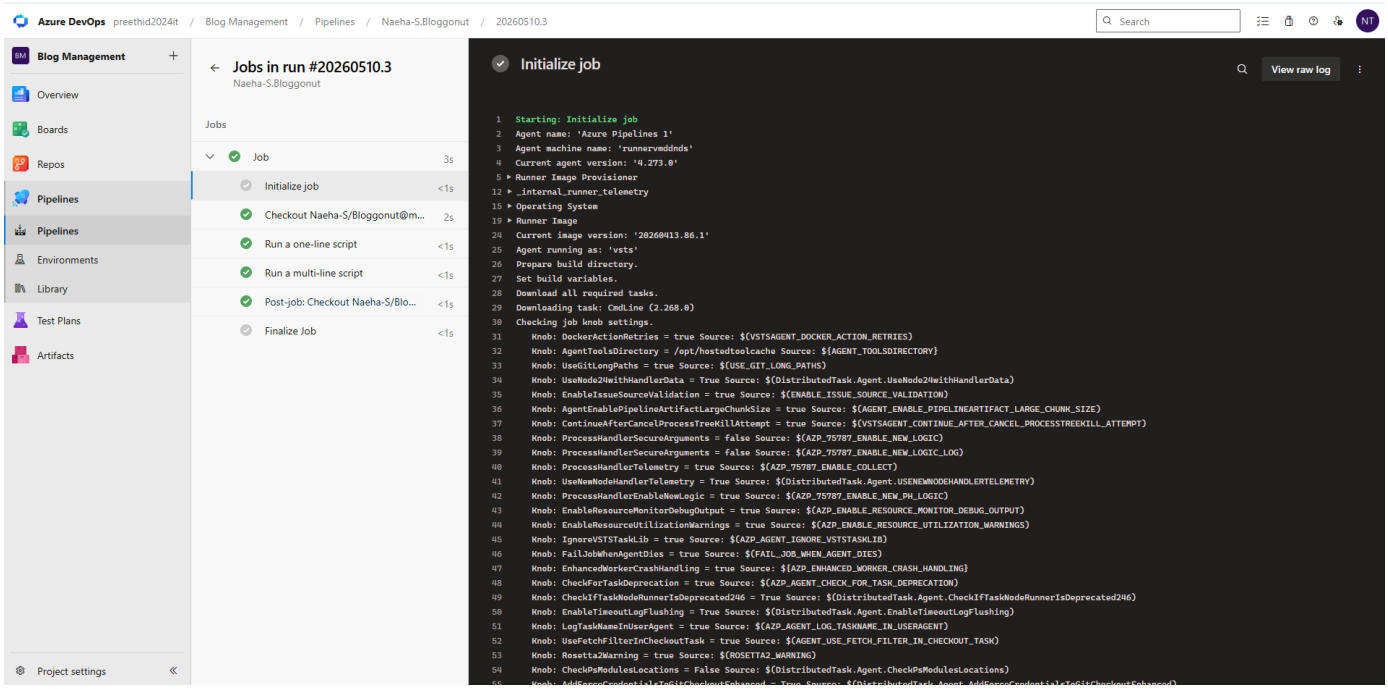
yml Code

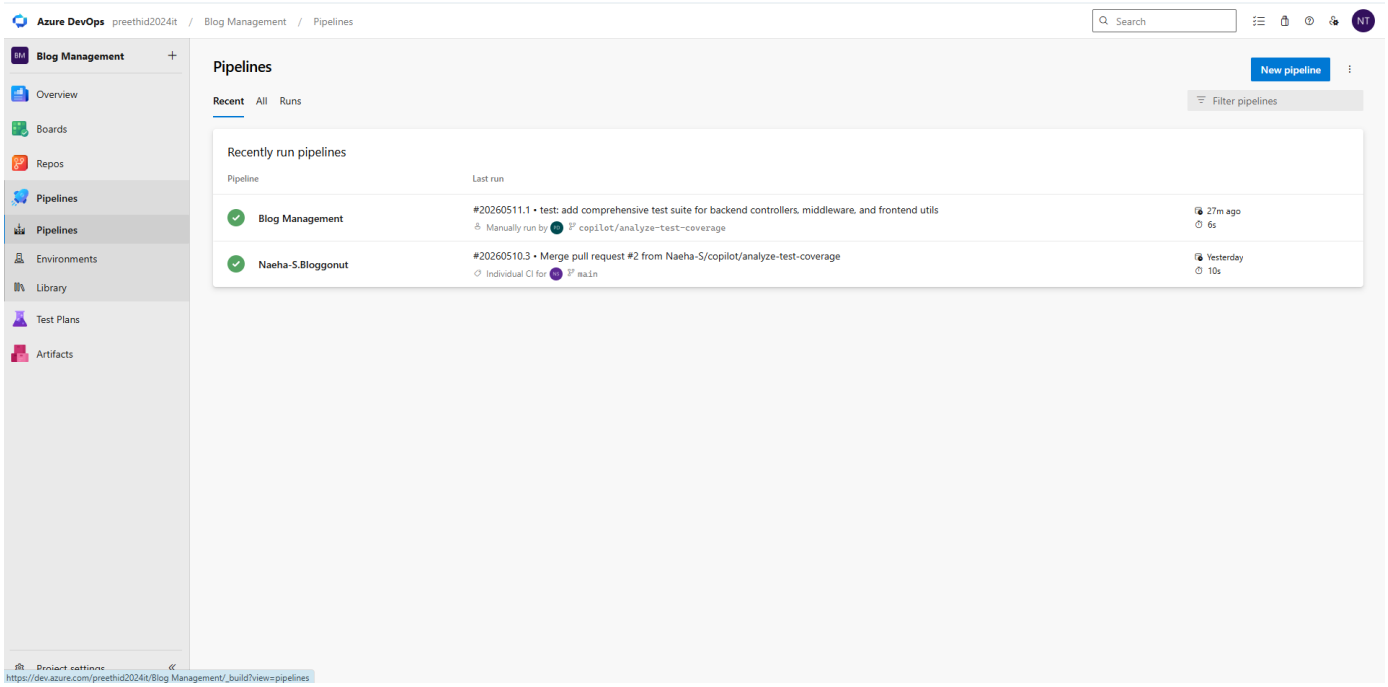
```
trigger:     -
main

pool:   vmImage: 'ubuntu-
latest'

variables:
  NODE_VERSION: '20.19.x'
  NPM_CACHE: '$(Pipeline.Workspace)/.npm'

steps:

#   Install Node.js (UPDATED TASK)
- task: UseNode@1    inputs:
    version: $(NODE_VERSION)
displayName: 'Install Node.js'

#   Cache npm (FIXED PATH) -
task: Cache@2    inputs:
    key: 'npm | "$(Agent.OS)" | **/package-lock.json'
path: $(NPM_CACHE)    displayName: 'Cache npm'

#   Install Frontend Dependencies
- script: |
    npm config set cache $(NPM_CACHE) --global
npm ci    workingDirectory: frontend
displayName: 'Install Frontend Dependencies'

#   Build Frontend (Vite) -
script: npm run build
workingDirectory: frontend
displayName: 'Build Frontend'

#   Frontend Tests (optional)
- script: npm test --if-present
  workingDirectory: frontend
  displayName: 'Run Frontend Tests'

#   Install Backend Dependencies
- script: |
    npm config set cache $(NPM_CACHE) --global
npm ci    workingDirectory: backend
displayName: 'Install Backend Dependencies'
```

```
#   Backend Tests
- script: npm test --if-present
  workingDirectory: backend
  displayName: 'Run Backend Tests'

#   Verify Frontend Build (fail early)
- script: |     if [ ! -d "frontend/dist" ]; then
  echo "Build failed: dist folder not found"
  exit 1     fi
  displayName: 'Verify Frontend Build'

#   Archive Frontend -
task: ArchiveFiles@2
inputs:
   rootFolderOrFile: 'frontend/dist'
includeRootFolder: false     archiveType:
'zip'
   archiveFile: '$(Build.ArtifactStagingDirectory)/frontend.zip'
displayName: 'Archive Frontend'

#   Archive Backend -
task: ArchiveFiles@2
inputs:
   rootFolderOrFile: 'backend'
includeRootFolder: false     archiveType:
'zip'
   archiveFile: '$(Build.ArtifactStagingDirectory)/backend.zip'
displayName: 'Archive Backend'

#   Publish Artifacts
- task: PublishBuildArtifacts@1
inputs:
   PathtoPublish: '$(Build.ArtifactStagingDirectory)'
ArtifactName: 'drop'   displayName: 'Publish
Artifacts'
```

3. Run and Monitor Pipeline:
    - Commit changes to the main branch of your repository to trigger the pipeline in Azure DevOps.
    - Monitor the logs in the Azure DevOps portal to view logs, errors, or success messages and ensure everything runs smoothly.

Pipeline


Azure DevOps preethid2024it / Blog Management / Pipelines / Naeha-S.Bloggonut / 20260510.3
Search
Blog Management
Overview
Boards
Repos
Pipelines
Pipelines
Environments
Library
Test Plans
Artifacts
Project settings
Jobs in run #20260510.3
Naeha-S.Bloggonut
Jobs
Job 3s
Initialize job <1s
Checkout Naeha-S/Bloggonut@m... 2s
Run a one-line script <1s
Run a multi-line script <1s
Post-job: Checkout Naeha-S/Blo... <1s
Finalize Job <1s
Initialize job
View raw log
Starting: Initialize job
Agent name: 'Azure Pipelines 1'
Agent machine name: 'runnervmddnds'
Current agent version: '4.273.0'
Runner Image Provisioner
_internal_runner_telemetry
Operating System
Runner Image
Current image version: '20260413.86.1'
Agent running as: 'vsts'
Prepare build directory.
Set build variables.
Download all required tasks.
Downloading task: CmdLine (2.268.0)
Checking job knob settings.
Knob: DockerActionRetries = true Source: $(VSTSAGENT_DOCKER_ACTION_RETRIES)
Knob: AgentToolsDirectory = /opt/hostedtoolcache Source: ${AGENT_TOOLSDIRECTORY}
Knob: UseGitLongPaths = true Source: $(USE_GIT_LONG_PATHS)
Knob: UseNode24withHandlerData = True Source: $(DistributedTask.Agent.UseNode24withHandlerData)
Knob: EnableIssueSourceValidation = true Source: $(ENABLE_ISSUE_SOURCE_VALIDATION)
Knob: AgentEnablePipelineArtifactLargeChunkSize = true Source: $(AGENT_ENABLE_PIPELINEARTIFACT_LARGE_CHUNK_SIZE)
Knob: ContinueAfterCancelProcessTreeKillAttempt = true Source: $(VSTSAGENT_CONTINUE_AFTER_CANCEL_PROCESSTREEKILL_ATTEMPT)
Knob: ProcessHandlerSecureArguments = false Source: $(AZP_75787_ENABLE_NEW_LOGIC)
Knob: ProcessHandlerSecureArguments = false Source: $(AZP_75787_ENABLE_NEW_LOGIC_LOG)
Knob: ProcessHandlerTelemetry = true Source: $(AZP_75787_ENABLE_COLLECT)
Knob: UseNewNodeHandlerTelemetry = True Source: $(DistributedTask.Agent.USENEWNODEHANDLERTELEMETRY)
Knob: ProcessHandlerEnableNewLogic = true Source: $(AZP_75787_ENABLE_NEW_PH_LOGIC)
Knob: EnableResourceMonitorDebugOutput = true Source: $(AZP_ENABLE_RESOURCE_MONITOR_DEBUG_OUTPUT)
Knob: EnableResourceUtilizationWarnings = true Source: $(AZP_ENABLE_RESOURCE_UTILIZATION_WARNINGS)
Knob: IgnoreVSTSTaskLib = true Source: $(AZP_AGENT_IGNORE_VSTSTASKLIB)
Knob: FailJobWhenAgentDies = true Source: $(FAIL_JOB_WHEN_AGENT_DIES)
Knob: EnhancedWorkerCrashHandling = true Source: $(AZP_ENHANCED_WORKER_CRASH_HANDLING)
Knob: CheckForTaskDeprecation = true Source: $(AZP_AGENT_CHECK_FOR_TASK_DEPRECATION)
Knob: CheckIfTaskNodeRunnerIsDeprecated246 = True Source: $(DistributedTask.Agent.CheckIfTaskNodeRunnerIsDeprecated246)
Knob: EnableTimeoutLogFlushing = True Source: $(DistributedTask.Agent.EnableTimeoutLogFlushing)
Knob: LogTaskNameInUserAgent = true Source: $(AZP_AGENT_LOG_TASKNAME_IN_USERAGENT)
Knob: UseFetchFilterInCheckoutTask = true Source: $(AGENT_USE_FETCH_FILTER_IN_CHECKOUT_TASK)
Knob: Rosetta2Warning = true Source: $(ROSETTA2_WARNING)
Knob: CheckPsModulesLocations = False Source: $(DistributedTask.Agent.CheckPsModulesLocations)


Azure DevOps preethid2024it / Blog Management / Pipelines
Search
Blog Management
Overview
Boards
Repos
Pipelines
Pipelines
Environments
Library
Test Plans
Artifacts
Pipelines
New pipeline
Recent All Runs
Filter pipelines
Recently run pipelines
Pipeline
Last run
Blog Management
#20260511.1 • test: add comprehensive test suite for backend controllers, middleware, and frontend utils
Manually run by
copilot/analyze-test-coverage
27m ago
6s
Naeha-S.Bloggonut
#20260510.3 • Merge pull request #2 from Naeha-S/copilot/analyze-test-coverage
Individual CI for
main
Yesterday
10s
https://dev.azure.com/preethid2024it/Blog Management/_build?view=pipelines

Result:

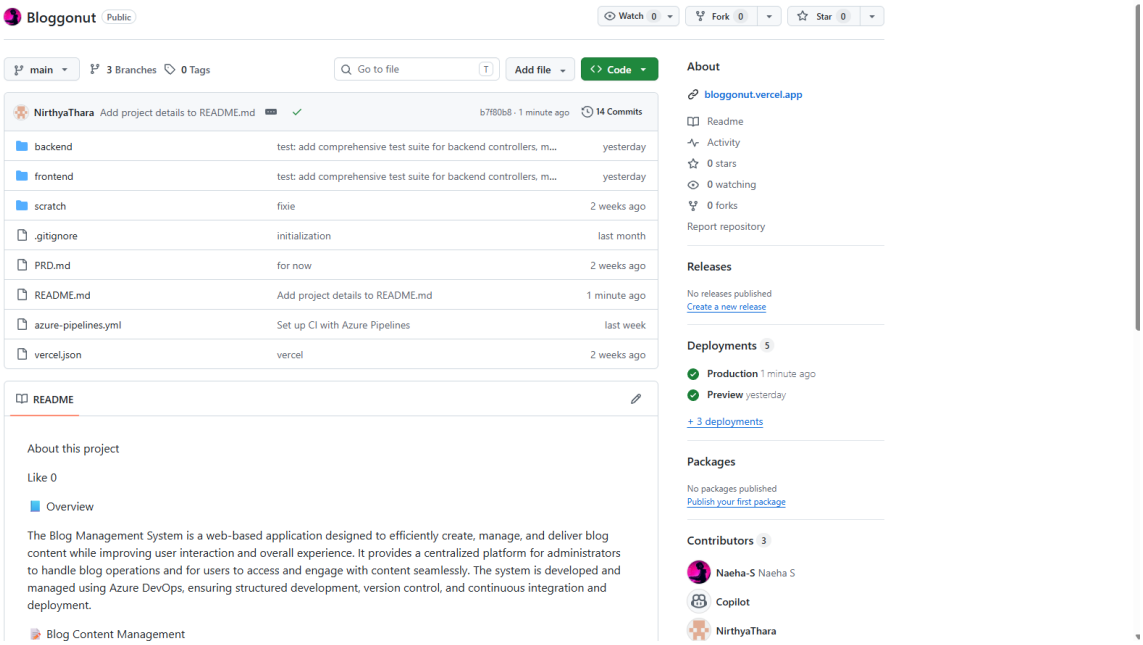
Successfully created the Azure Load Testing resource and executed a load test to assess the performance of the specified endpoint and also demonstrated pipelines in azure devops.

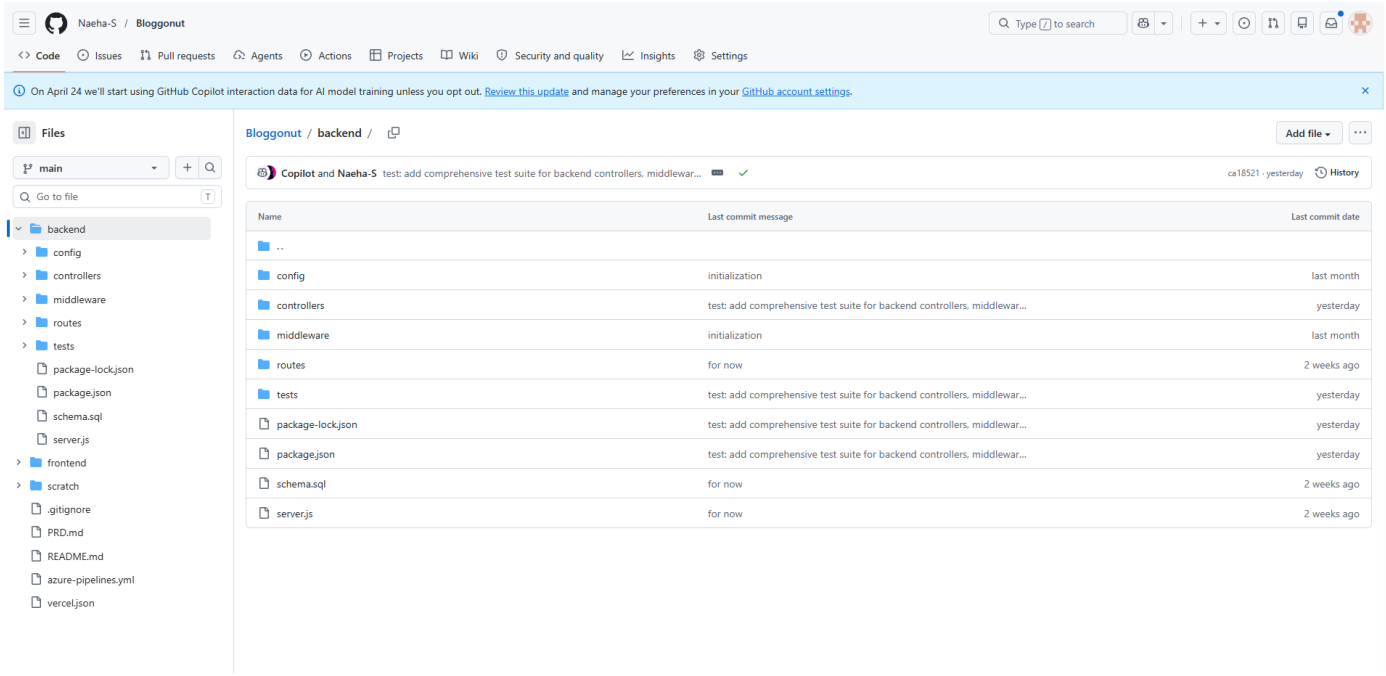
EXP NO: 10	GITHUB: PROJECT STRUCTURE & NAMING CONVENTIONS

Aim:

To provide a clear and organized view of the project's folder structure and file naming conventions, helping contributors and users easily understand, navigate, and extend the Blog Management project.

GitHub Project Structure





Result:

The GitHub repository clearly displays the organized project structure and consistent naming conventions, making it easy for users and contributors to understand and navigate the codebase.